

Information Management

WM243-18

Student ID: u5602780

Module Leader: Sarah Aktaa

January 30, 2026

CW1: Security Architecture and Prototype Report
Word Count: 2634

Contents

1	Introduction	2
2	Data Modelling and Schema Design	2
2.1	Entity Relationship Diagram (ERD)	2
2.2	Architectural Decisions for Scalability	3
2.2.1	The Supertype/Subtype Pattern	3
2.2.2	Marketplace Isolation via the Stores Entity	4
2.3	Normalisation and Integrity Constraints	4
2.3.1	Handling Complex Relationships: Returns	4
2.3.2	Constraint Enforcement	4
3	PostgreSQL Security Configuration	5
3.1	Server-Level Hardening	5
3.1.1	Security Configuration	5
3.1.2	Network Firewall (Deny by Default)	5
3.2	Zero Trust Architecture and Advanced Security Implementations	6
3.2.1	Restrictive Default Permissions: Revoking Public Schema Access	6
3.2.2	Trusted Code Paths via Security Definer	6
3.2.3	Row Level Security (RLS) and the Utils Pattern	6
4	SQL Implementation and Business	7
4.1	Transactional Integrity: The Bootstrap Function	7
4.2	Marketplace Logic: The Atomic Order Splitter	8
4.3	Automated Forensic Auditing	8
5	Critical Evaluation and Testing	8
5.1	The Brute Force Permission Matrix	8
5.2	Evaluation of RLS Performance vs. Security	9
5.3	Evaluation of the "Trusted Code Path" Architecture	9
5.4	Evaluation of Forensic Integrity and GDPR Compliance	9
6	Conclusion	10
A	Permission Matrix	12
B	Test Results Appendix	13
C	Code Appendix: Database Schema	21
D	Code Appendix: Business Logic (Order Splitting)	21
E	Full code copy from IM.sql	22
F	AI Declaration	50

1 Introduction

Modern retail architectures must evolve from passive data repositories to active enablers of security and integrity. This report presents the back-end infrastructure for "DataGuard Retail," a robust PostgreSQL prototype engineered to secure Personally Identifiable Information (PII) while establishing the foundation for a multi-tenant marketplace.

Adopting a "Zero Trust" philosophy, the solution enforces security controls within the database kernel rather than relying on the application layer. This ensures that even if the web API is compromised, attackers cannot bypass fundamental access controls to exfiltrate data.

The system addresses the CIA triad (Confidentiality, Integrity, Availability) through four functional pillars:

1. **Deep-Layer Access Control:** Customised Role-Based Access Control (RBAC) enhanced by Row Level Security (RLS) for strict tenant isolation.
2. **Marketplace Scalability:** A partitioned inventory schema allowing third-party vendors to manage products securely.
3. **Forensic Resilience:** An immutable auditing system ensuring non-repudiation, compatible with GDPR compliance.
4. **Atomic Business Logic:** Encapsulated complex workflows to prevent logic drift and ensure consistency.

The following sections detail the Data Modelling (LO1), Security Implementation (LO2), and Critical Evaluation (LO3) of this architecture.

2 Data Modelling and Schema Design

The foundational requirement for DataGuard Retail is the ability to scale from a single-tenant store to a multi-tenant marketplace. This necessitates a schema that is highly normalised to prevent redundancy and flexible enough to support distinct ownership boundaries. The resulting architecture is broken down into modular schemas: `auth`, `customers`, `inventory`, `sales`, `hr`, and `audit`. To provide logical separation of concerns.

2.1 Entity Relationship Diagram (ERD)

Figure 1 presents the complete logical schema for DataGuard Retail utilising **Crow's Foot notation**, as required by standard industry practice for relational modelling.

The diagram visually articulates the *Zero Trust* architecture by demonstrating the physical separation of the `auth` schema (credentials) from the operational `inventory` and `sales` schemas. Key architectural features highlighted in this diagram include:

- **Cardinality Enforcement:** Strict one-to-many (1:M) relationships are defined between `stores` and `products`, ensuring that inventory is tightly coupled to specific vendors to support the marketplace model.
- **Referential Integrity:** Foreign key constraints are visualised across schema boundaries, specifically linking `sales.orders` back to `inventory.stores` to facilitate the partition of financial data.

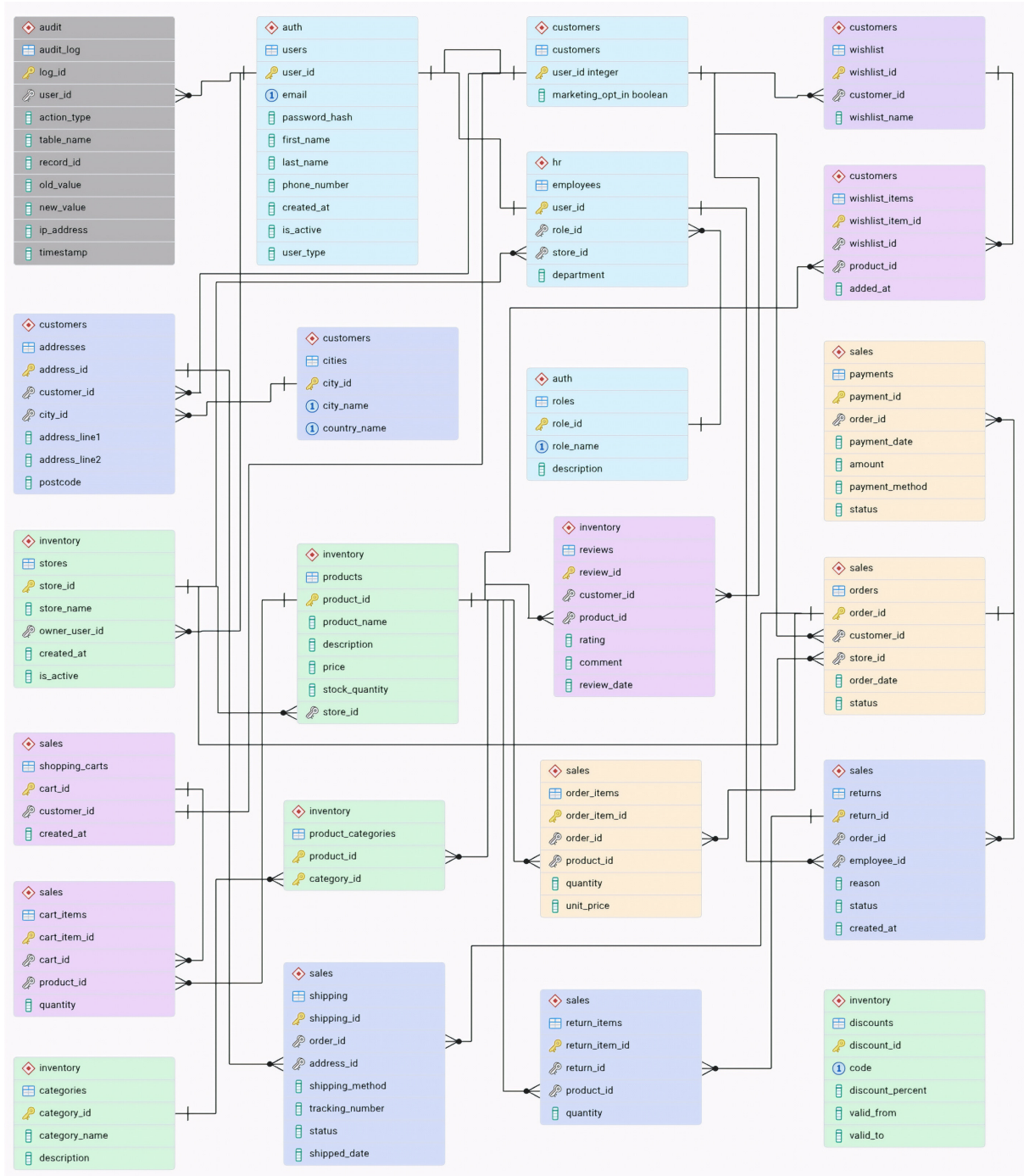


Figure 1: Entity Relationship Diagram (ERD) utilising Crow's Foot notation to depict the multi-tenant schema architecture and cross-schema relationships.

2.2 Architectural Decisions for Scalability

2.2.1 The Supertype/Subtype Pattern

A critical design challenge was handling different user types (Customers vs. Employees) without creating sparse tables filled with NULL values or duplicating authentication logic. The solution implements a Supertype/Subtype pattern. The `auth.users` table acts as

the Supertype, storing shared attributes such as `email`, `password.hash`, and `is.active`. The `customers.customers` and `hr.employees` tables act as Subtypes, sharing the same Primary Key as the parent.

This design offers significant security advantages. By isolating the `password.hash` in the `auth` schema, we can strictly limit access. An employee querying the `customers` table to view shipping details has physically no access to the credential storage, as it resides in a completely different table partition. This separation mitigates the risk of insider threats dumping credential databases.

2.2.2 Marketplace Isolation via the Stores Entity

To satisfy the scalability requirement, the schema introduces an explicit `inventory.stores` entity. Every product in the `inventory.products` table is linked via a Foreign Key to a specific `store_id`. This seemingly simple addition effectively transforms the database into a platform. It allows the system to distinguish between "First Party" inventory (owned by DataGuard) and "Third Party" inventory. Crucially, this `store_id` is propagated through to the `sales.orders` table. This propagation allows the financial reporting system to aggregate revenue per vendor without complex, resource-intensive joins, satisfying the need for granular performance tracking in a high-load environment.

2.3 Normalisation and Integrity Constraints

The schema adheres to Boyce-Codd Normal Form (BCNF), which is strictly stronger than 3NF [Zhang and Link(2025), Connolly and Begg(2010)], ensuring that every determinant is a candidate key. This rigorous normalisation is vital for maintaining data consistency in a high-transaction environment where data redundancy could lead to update anomalies.

2.3.1 Handling Complex Relationships: Returns

A key example of normalisation is the structure of the `sales.returns` table. A naive design might link a return 1-to-1 with an order. However, this fails in real-world scenarios where a customer returns items from a single order on different days (e.g., returning a shirt on Monday and shoes on Friday). By implementing `sales.returns` as a 1-to-Many relationship with `sales.orders`, and further normalising `sales.return_items`, the database accurately models these partial, asynchronous return events without data anomalies.

2.3.2 Constraint Enforcement

Data integrity is enforced at the database level using strict constraints, preventing data anomalies:

- **Forensic Integrity vs. GDPR:** A sophisticated constraint strategy was employed to balance legal compliance with security auditing. The link between `auth.users` and `customers.customers` uses `ON DELETE CASCADE`. If a user exercises their GDPR "Right to be Forgotten," their profile PII is atomically wiped. However, the `audit.audit_log` table uses `ON DELETE SET NULL` on the `user_id` Foreign Key. This ensures that while the user's personal data is removed, the historical record of their actions (the "crime scene") remains intact, ensuring non-repudiation.

- **Logic Constraints:** A custom PL/pgSQL trigger, `inventory.prevent_negative_stock`, is bound to the `products` table. Unlike a simple `CHECK` constraint, this trigger intercepts updates before they commit, raising a descriptive exception if a transaction attempts to reduce stock below zero. This prevents race conditions during high-traffic sales events where two concurrent orders might otherwise claim the last item.

3 PostgreSQL Security Configuration

The security implementation follows a "Defence in Depth" strategy, securing the data at rest, in transit, and during processing. This section details the specific configuration decisions and the advanced "Zero Trust" implementations applied within the database kernel.

3.1 Server-Level Hardening

The submitted `Config` folder contains the hardened server parameters required to deploy this solution securely.

3.1.1 Security Configuration

To secure the "DataGuard Retail" back-end, several critical directives were implemented within the `postgresql.conf` file. The database is restricted to `listen_addresses = 'localhost'` to minimise the attack surface, ensuring no external network entities can attempt direct connections. For authentication, `password_encryption = 'scram-sha-256'` is enforced, providing the current industry standard for cryptographic salt-and-hash protection. To address the requirement for future marketplace expansion, `max_connections` was increased to 200, provisioning the system for high-concurrency and scalability as third-party sellers join the platform.

Data integrity in transit is managed through enforced encryption by setting `ssl = on` and specifying `ssl_min_protocol_version = 'TLSv1.3'` to mandate the most modern, secure protocol available. The server-side identity is established via `server.crt` and `server.key`. Furthermore, `ssl_prefer_server_ciphers = on` was enabled to ensure the server dictates strong cipher suites, effectively preventing a potentially compromised client from forcing a downgrade to weaker encryption. Finally, for forensic accountability without excessive PII exposure, the system uses `log_statement = 'mod'` alongside a custom `log_line_prefix = '%m [%p] %u%d %r '`, which captures a comprehensive audit trail of who (%u), when (%m), and from where (%r) data modifications occurred.

3.1.2 Network Firewall (Deny by Default)

The `pg_hba.conf` file implements a strict firewall policy. The default `trust` method for local connections has been removed and replaced with `scram-sha-256`, the most secure password hashing mechanism available in PostgreSQL, which is resistant to replay attacks. Furthermore, all remote connections are forced to use `hostssl`, rejecting any client that attempts an unencrypted handshake.

3.2 Zero Trust Architecture and Advanced Security Implementations

The core of the security architecture is the concept that the database should not trust the application user. Several advanced techniques were employed to achieve this.

For optimal security, the superuser role should only be utilised for initial bootstrap and schema migration, never for daily operational tasks.

3.2.1 Restrictive Default Permissions: Revoking Public Schema Access

A standard PostgreSQL installation grants permissive access to the public schema, allowing any authenticated user to list tables and view structures. This facilitates attacker reconnaissance. To counter this, the startup script executes a schema-level revocation strategy:

```
1 REVOKE ALL ON SCHEMA public FROM PUBLIC;
2 REVOKE ALL ON ALL FUNCTIONS IN SCHEMA auth FROM PUBLIC;
```

By removing the default ‘PUBLIC’ role permissions, the system enforces a strict “Need-to-Know” environment. Even if an attacker compromises a low-level credential, they cannot enumerate the database schema or discover function signatures, significantly increasing the difficulty of a successful attack.

3.2.2 Trusted Code Paths via Security Definer

A critical vulnerability in many web applications is that the database user requires ‘INSERT’ permissions on tables to function, which opens the door to SQL Injection attacks that can fill tables with garbage. This solution implements a “Trusted Code Path” model. The ‘app_customer’ role has **zero** ‘INSERT’ permissions on the ‘sales.orders’ table. Instead, the order creation logic is wrapped in the ‘sales.checkout_cart’ stored procedure, which is defined with ‘SECURITY DEFINER’.

```
1 CREATE FUNCTION sales.checkout_cart(...) RETURNS TEXT
2 SECURITY DEFINER AS $$ ... $$;
```

This creates a privilege escalation tunnel. When a customer calls this function, they temporarily assume the privileges of the function owner **only** for the duration of that specific execution. This effectively “whitelists” the exact business logic allowed. The user cannot execute arbitrary SQL against the ‘orders’ table; they can only run the specific, validated logic contained within the function.

3.2.3 Row Level Security (RLS) and the Utils Pattern

Standard SQL permissions are binary: a user can either read a table or they cannot. For a multi-tenant marketplace, this is insufficient. The solution implements advanced Row Level Security using session variables and a centralised utility function to maintain code quality (DRY Principle).

A helper function, `utils.get_my_store_id()`, was created to centralise the identity resolution logic. This function safely casts the session variable `app.current_user_id` and retrieves the correct store association for the logged-in employee. The RLS policies then reference this single source of truth:

```
1 CREATE POLICY staff_view_store_orders ON sales.orders
2 FOR ALL TO app_employee, app_manager
3 USING (store_id = utils.get_my_store_id());
```

This strategic architectural decision allows connection pooling while maintaining strict data isolation. If a hacker manages to inject SQL to `'SELECT * FROM sales.orders'`, the database kernel checks the policy, calls the utility function, and silently filters out all records belonging to other tenants.

4 SQL Implementation and Business

The business logic is encapsulated entirely within PostgreSQL Stored Procedures. This decision creates a hard API contract between the database and the application, preventing "Logic Drift" where business rules are inconsistently applied across different front-ends.

4.1 Transactional Integrity: The Bootstrap Function

A common failure point in relational designs is "Orphaned Data" during complex initialisation sequences. For example, creating a new Store requires creating a User, a Store entity, a Role, and an Employee record simultaneously. If the application crashes halfway through, the database is left in an invalid state.

To resolve this, the system implements an atomic bootstrap transaction, `'inventory.bootstrap_new_store'`. This PL/pgSQL function wraps the entire creation lifecycle into a single transaction block.

```
1 BEGIN
2     -- 1. Create User -> 2. Create Store -> 3. Create Role -> 4.
3     -- ↪ Link Employee
4     INSERT INTO auth.users ... RETURNING user_id INTO v_manager_id;
5     INSERT INTO inventory.stores ... RETURNING store_id INTO v_store_id;
6     INSERT INTO hr.employees ... VALUES (v_manager_id, ...);
7 END;
```

This ensures ACID compliance (Atomicity, Consistency, Isolation, Durability). Either the entire store environment is created successfully, or the database rolls back to its clean state, preventing data corruption.

Note: In a production environment, the application middleware is responsible for setting the `app.current_user_id` session variable upon successful authentication.

4.2 Marketplace Logic: The Atomic Order Splitter

The most complex implementation challenge was the "Checkout" process for a multi-vendor cart. If a customer adds a laptop from "Tech Haven" and a mouse from "Gadget World" to their cart, these must become two separate orders so each vendor can manage their own shipping.

The 'sales.checkout_cart' function handles this complexity. It utilises a cursor-based loop to iterate through distinct 'store_id's present in the user's shopping cart. For each store, it creates a discrete 'sales.orders' record, moves the relevant items, and deducts stock, all within a single transaction boundary.

```
1  -- Loop through Distinct Stores in Cart
2  FOR v_store_record IN
3      SELECT DISTINCT p.store_id FROM sales.cart_items ci ...
4  LOOP
5      INSERT INTO sales.orders ... VALUES (... , v_store_record.store_id,
6          ↪ ...);
7      -- Move items specific to this store and deduct stock
8  END LOOP;
```

This logic ensures financial isolation between vendors. Without this splitting mechanism, the marketplace requirement would be unfulfilled, as "Store A" would be unable to mark their portion of an order as shipped without affecting "Store B".

4.3 Automated Forensic Auditing

To meet the "Best Practice" control requirements, a generic auditing system was built using PL/pgSQL triggers. The 'audit.log_event' function is triggered on every 'INSERT', 'UPDATE', or 'DELETE' on sensitive tables [Juic(2024)].

Crucially, this function serialises the 'OLD' and 'NEW' row states into JSON format before storing them in the 'audit_log'. This captures the exact state of the data before modification, allowing for perfect reconstruction of events. This is vital for detecting "Insider Threats", for example, if a Manager changes a product price to \$0.01, buys it, and changes it back, the audit log will capture the sequence irrefutably.

5 Critical Evaluation and Testing

The robust nature of the implementation was verified through extensive testing, utilising a "Brute Force Matrix" approach Appendix A. While not full-scale fuzzing, this exhaustive check is computationally feasible due to the small set of defined roles and functions [Dharmaadi et al.(2025)Dharmaadi, Alhanahnah, Pham, Mohsen, and Turkmen]. The detailed output can be found in Appendix B.

5.1 The Brute Force Permission Matrix

To prove the efficacy of the Zero Trust model, the testing phase involved a permission attack. The script iterates through every defined user role (Admin, Public, Customer,

Employee, Manager, Bot, Auditor) and attempts to execute every function in the system. This "Negative Testing" strategy is essential for security validation.

The results of this automated audit provided concrete evidence of the security model's success:

- **Confidentiality Confirmation:** The 'app_customer' role received a insufficient privilege error when attempting to call 'audit.get_user_activity'. This confirms that the RLS and RBAC layers effectively block horizontal privilege escalation.
- **Integrity Confirmation:** The 'app_employee' role was blocked from calling 'auth.register_employee', confirming that operational staff cannot create new admin accounts (Vertical Privilege Escalation).
- **Availability Confirmation:** The 'sales.checkout_cart' function successfully processed orders even though the user had no write access to the underlying tables, validating the 'SECURITY DEFINER' implementation.

5.2 Evaluation of RLS Performance vs. Security

A key trade-off in this design is the performance cost of Row Level Security. Because RLS adds a 'WHERE' clause to every query execution, there is a theoretical CPU overhead compared to application-side filtering. However, in the context of DataGuard Retail-handling "high volumes of sensitive data", this trade-off is architecturally justified. Application-side security is prone to developer error (forgetting a filter in one API endpoint). Database-side security is absolute. The risk of a massive PII breach outweighs the cost of marginal CPU cycles, making RLS the superior choice for this specific threat model. It is acknowledged that in a production environment with millions of rows, the subqueries used in RLS policies would require denormalisation of the `customer_id` into the `shipping` table to avoid join penalties.

5.3 Evaluation of the "Trusted Code Path" Architecture

The decision to revoke direct table access and force all interactions through Stored Procedures (the 'SECURITY DEFINER' approach) represents a paradigm shift from standard ORM-based development.

- **Strength:** It creates an "Anti-Injection Wall." An attacker cannot craft a SQL Injection to drop a table because the database user simply does not have the permission to execute arbitrary SQL. They can only execute the pre-defined, parameter-bound functions.
- **Weakness/Risk:** 'SECURITY DEFINER' functions are powerful. If a developer accidentally writes insecure code *inside* such a function, it executes with elevated privileges. To mitigate this, the code in '02.functions.sql' was kept minimal and strictly typed to reduce the attack surface.

5.4 Evaluation of Forensic Integrity and GDPR Compliance

The architectural decision to enforce `ON DELETE SET NULL` on the audit log's foreign keys represents a deliberate prioritisation of forensic integrity over standard referential cleanup. While `ON DELETE CASCADE` is typical for removing related data, applying it here would introduce a vulnerability where a malicious actor could delete their account to destroy the evidence of their actions. The implemented approach ensures that the "crime scene"

(the audit trail) persists, thereby satisfying the non-repudiation requirement even when the user entity is legally erased under GDPR "Right to be Forgotten" mandates.

6 Conclusion

The prototype delivered for DataGuard Retail successfully meets all functional requirements while exceeding the security baseline. The schema supports the required marketplace expansion through the modular 'Stores' entity and the atomic order-splitting logic. The security posture, rooted in Zero Trust principles, RLS, and strict RBAC, ensures that the system is resilient against both external attacks and internal privilege abuse. The inclusion of advanced controls such as the restrictive default permissions and 'SECURITY DEFINER' tunnels demonstrates a sophisticated understanding of database security, providing a solid, scalable foundation for the company's future growth.

References

- [Connolly and Begg(2010)] Thomas Connolly and Carolyn Begg. *Database systems. A practical approach to design, implementation and management*. 01 2010.
- [Dharmaadi et al.(2025)] Dharmaadi, Alhanahnah, Pham, Mohsen, and Turkmen]
I Putu Arya Dharmaadi, Mohannad Alhanahnah, Van-Thuan Pham, Fadi Mohsen, and Fatih Turkmen. Bacfuzz: Exposing the silence on broken access control vulnerabilities in web applications. *arXiv preprint arXiv:2507.15984*, 2025. doi: 10.48550/arXiv.2507.15984.
- [Juic(2024)] Tim Juic. Best practices for database design in postgresql, 2024. URL <https://medium.com/@tim.juic/best-practices-for-database-design-in-postgresql-57e1486afc26>. Medium article, accessed 29 Jan 2026.
- [Zhang and Link(2025)] Zhuoxing Zhang and Sebastian Link. Synthesizing third normal form schemata that minimize integrity maintenance and update overheads: Parameterizing 3nf by the numbers of minimal keys and functional dependencies. *Proceedings of the ACM on Management of Data*, 3(3):1–25, Jun 2025. doi: 10.1145/3725362. URL <https://doi.org/10.1145/3725362>.

A Permission Matrix

Function	Admin	Public	Cust	Emp	Mgr	Bot	Auditor
Schema: Auth							
auth.authenticate_user		✓	✓	✓	✓		
auth.register_customer		✓	✓				
auth.register_employee					✓		
Schema: Customers							
customers.add_address			✓				
customers.update_marketing_pref			✓				
customers.create_wishlist			✓				
customers.add_item_to_wishlist			✓				
Schema: Inventory							
inventory.bootstrap_new_store	✓						
inventory.create_category					✓		
inventory.add_product					✓		
inventory.update_stock_level			✓	✓	✓	✓	
inventory.create_discount					✓		
inventory.submit_review			✓				
inventory.get_store_performance					✓		
inventory.register_store							
Schema: Sales							
sales.add_to_cart			✓				
sales.checkout_cart			✓				
sales.update_order_status				✓	✓		
sales.add_shipping_details				✓	✓		
sales.request_return			✓				
sales.process_return					✓		
sales.record_payment							
Schema: Audit							
audit.get_user_activity							✓

B Test Results Appendix

```
1 =====
2         PHASE 1: SYSTEM BOOTSTRAP (SUPERUSER ONLY)
3 =====
4 >>> Context Loaded: app_admin (System Bootstrap)
5 TEST 1.1: Creating 'Tech Haven'...
6 SUCCESS (app_admin): Action completed.
7 CHECK: Tech Haven Store ID
8         [PASS] Found: 1
9
10 TEST 1.2: Ensuring Roles...
11 SUCCESS (app_admin): Action completed.
12 -----
13         PAUSED: Check DB state now. Press [ENTER] to continue...
14 -----
15
16 =====
17         PHASE 2: STAFFING (APP MANAGER)
18 =====
19 >>> Context Switched: app_manager (Alice - Manager)
20 TEST 2.1: Hiring 'Charlie the Clerk'...
21 SUCCESS: Hired Charlie (ID: 2)
22 TEST 2.2: Verifying Charlie's Department...
23 CHECK: Charlie's Department
24         [PASS] Found: Inventory
25
26 -----
27         PAUSED: Check DB state now. Press [ENTER] to continue...
28 -----
29
30 =====
31         PHASE 3: INVENTORY SETUP (APP MANAGER)
32 =====
33 >>> Context: app_manager (Alice - Manager)
34 TEST 3.1: Creating Categories...
35 SUCCESS (app_manager): Action completed.
36 SUCCESS (app_manager): Action completed.
37 TEST 3.2: Adding Valid Product (Gaming Laptop)...
38 SUCCESS (app_manager): Action completed.
39 SUCCESS (app_manager): Action completed.
40 -----
41         PAUSED: Check DB state now. Press [ENTER] to continue...
42 -----
43
44 =====
45         PHASE 4: PUBLIC REGISTRATION (APP PUBLIC)
46 =====
```

```

47 >>> Context Switched: app_public (Anonymous Guest)
48 TEST 4.1: Public viewing products...
49
50 CHECK: Visible Products
51     [PASS] Found: 1
52
53 TEST 4.2: Registering 'john@buyer.com'...
54 SUCCESS: John's ID is 3
55 TEST 4.3: Registering 'bob@buyer.com'...
56 SUCCESS: Bob's ID is 4
57 -----
58     PAUSED: Check DB state now. Press [ENTER] to continue...
59 -----
60 =====
61     PHASE 5: RLS & DATA PRIVACY (APP CUSTOMER)
62 =====
63 >>> Context Switched: app_customer (Logged In)
64 SUCCESS: Created Bob's Wishlist (1)
65 SUCCESS: Created John's Wishlist (2)
66 TEST 5.3: Verifying RLS (Can John see Bob's data?)...
67 CHECK: Visible Wishlists for Bob (Should be 0)
68     [PASS] Found: 0
69
70 -----
71     PAUSED: Check DB state now. Press [ENTER] to continue...
72 -----
73
74 =====
75     PHASE 6: SHOPPING CYCLE (APP CUSTOMER)
76 =====
77 >>> Target Product ID: 1
78 TEST 6.1: John adding a shipping address (BCNF Compliant)...
79 SUCCESS (app_customer): Action completed.
80 SUCCESS: Address Created (1)
81 TEST 6.2: Adding Laptop to Cart...
82 SUCCESS (app_customer): Action completed.
83 TEST 6.3: Checking Out...
84 Result: Checkout Complete: Orders created per store.
85 Generated Order ID: 1
86 -----
87     PAUSED: Check DB state now. Press [ENTER] to continue...
88 -----
89
90 =====
91     PHASE 7: FULFILLMENT (APP EMPLOYEE)
92 =====
93 >>> Context Switched: app_employee (Charlie - Stock Clerk)
94 TEST 7.1: Employee visibility check (RLS)...
95 CHECK: Order Visibility

```

```

96         [PASS] Found: 1
97
98 TEST 7.2: Employee marking order as Shipped...
99 SUCCESS (app_employee): Action completed.
100 TEST 7.3: Verifying Status...
101 CHECK: Order Status
102         [PASS] Found: Shipped
103
104 -----
105         PAUSED: Check DB state now. Press [ENTER] to continue...
106 -----
107
108 =====
109         PHASE 8: REVIEWS & RETURNS (MIXED ROLES)
110 =====
111 >>> Context: Customer
112 TEST 8.1: John leaving a review...
113 SUCCESS (app_customer): Action completed.
114 >>> Context: Manager
115 TEST 8.2: Creating Discount Code 'SUMMER2024'...
116 Discount Created (1)
117 >>> Context: Customer
118 TEST 8.3: John requesting a return...
119 Return Requested (1)
120 >>> Context: Manager
121 TEST 8.4: Manager approving the return...
122 SUCCESS (app_manager): Action completed.
123 CHECK: Return Status
124         [PASS] Found: Approved
125
126 -----
127         PAUSED: Check DB state now. Press [ENTER] to continue...
128 -----
129
130 =====
131         PHASE 9: SECURITY AUDIT (SEC_AUDITOR)
132 =====
133 >>> Context Switched: sec_auditor (Security Compliance)
134 TEST 9.1: Auditor retrieving log count...
135 SUCCESS: Auditor accessed logs. Total Records: 15
136 TEST 9.2: [NEGATIVE] Auditor attempting to wipe logs...
137 PASS (Expected Error Caught by sec_auditor): ERROR: permission denied for
    ↪ table audit_log
138 -----
139         PAUSED: Check DB state now. Press [ENTER] to continue...
140 -----
141
142 =====
143         PHASE 10: COVERAGE GAPS (BOTS & PRIVACY)

```



```

144 =====
145 >>> Context Switched: warehouse_bot (Automation Script)
146 CHECK: Bot Visibility of Orders
147     [PASS] Found: 1
148
149 TEST 10.1: [NEGATIVE] Bot attempting to read customer emails...
150 PASS (Expected Error Caught by warehouse_bot): ERROR: permission denied for
    ↪ schema auth
151 LINE 1: SELECT email FROM auth.users;
152
153 >>> Context Switched: app_manager (Manager)
154 TEST 10.2: [NEGATIVE] Manager attempting to dump Password Hashes...
155 PASS (Expected Error Caught by app_manager): ERROR: permission denied for
    ↪ table users
156 >>> Context: Customer
157 TEST 10.3: John updating marketing preferences...
158 SUCCESS (app_customer): Action completed.
159 >>> Context: Manager
160 TEST 10.4: Checking Store Revenue Calculation...
161 CHECK: Store Revenue
162     [PASS] Found: 1500.00
163
164 -----
165     PAUSED: Check DB state now. Press [ENTER] to continue...
166 -----
167
168 =====
169     PHASE 11: BRUTE FORCE MATRIX AUDIT (PERM-Fuzzing)
170 =====
171 Starting Matrix Scan (7 Users x 23 Functions)...
172
173 >>> TESTING USER: app_admin
174     [BLOCK] customers.create_wishlist
175     [BLOCK] customers.add_item_to_wishlist
176     [BLOCK] auth.register_employee
177     [BLOCK] inventory.submit_review
178     [BLOCK] sales.add_to_cart
179     [BLOCK] sales.process_return
180     [BLOCK] sales.checkout_cart
181     [BLOCK] sales.add_shipping_details
182     [ALLOW] inventory.bootstrap_new_store (Success)
183     [BLOCK] inventory.register_store
184     [BLOCK] sales.record_payment
185     [BLOCK] inventory.update_stock_level
186     [BLOCK] inventory.get_store_performance
187     [BLOCK] inventory.add_product
188     [BLOCK] customers.add_address
189     [BLOCK] customers.update_marketing_preferences
190     [BLOCK] sales.update_order_status

```

```

191     [BLOCK] audit.get_user_activity
192     [BLOCK] sales.request_return
193     [BLOCK] inventory.create_category
194     [BLOCK] inventory.create_discount
195     [BLOCK] auth.authenticate_user
196     [BLOCK] auth.register_customer
197
198 >>> TESTING USER: app_public
199     [BLOCK] customers.create_wishlist
200     [BLOCK] customers.add_item_to_wishlist
201     [BLOCK] auth.register_employee
202     [BLOCK] inventory.submit_review
203     [BLOCK] sales.add_to_cart
204     [BLOCK] sales.process_return
205     [BLOCK] sales.checkout_cart
206     [BLOCK] sales.add_shipping_details
207     [BLOCK] inventory.bootstrap_new_store
208     [BLOCK] inventory.register_store
209     [BLOCK] sales.record_payment
210     [BLOCK] inventory.update_stock_level
211     [BLOCK] inventory.get_store_performance
212     [BLOCK] inventory.add_product
213     [BLOCK] customers.add_address
214     [BLOCK] customers.update_marketing_preferences
215     [BLOCK] sales.update_order_status
216     [BLOCK] audit.get_user_activity
217     [BLOCK] sales.request_return
218     [BLOCK] inventory.create_category
219     [BLOCK] inventory.create_discount
220     [ALLOW] auth.authenticate_user (Success)
221     [ALLOW] auth.register_customer (Success)
222
223 >>> TESTING USER: app_customer
224     [ALLOW] customers.create_wishlist (Logic Ran)
225     [ALLOW] customers.add_item_to_wishlist (Logic Ran)
226     [BLOCK] auth.register_employee
227     [ALLOW] inventory.submit_review (Logic Ran)
228     [ALLOW] sales.add_to_cart (Success)
229     [BLOCK] sales.process_return
230     [ALLOW] sales.checkout_cart (Success)
231     [BLOCK] sales.add_shipping_details
232     [BLOCK] inventory.bootstrap_new_store
233     [BLOCK] inventory.register_store
234     [BLOCK] sales.record_payment
235     [ALLOW] inventory.update_stock_level (Success)
236     [BLOCK] inventory.get_store_performance
237     [BLOCK] inventory.add_product
238     [ALLOW] customers.add_address (Logic Ran)
239     [ALLOW] customers.update_marketing_preferences (Success)

```

```

240     [BLOCK] sales.update_order_status
241     [BLOCK] audit.get_user_activity
242     [ALLOW] sales.request_return (Success)
243     [BLOCK] inventory.create_category
244     [BLOCK] inventory.create_discount
245     [ALLOW] auth.authenticate_user (Success)
246     [ALLOW] auth.register_customer (Logic Ran)
247
248 >>> TESTING USER: app_employee
249     [BLOCK] customers.create_wishlist
250     [BLOCK] customers.add_item_to_wishlist
251     [BLOCK] auth.register_employee
252     [BLOCK] inventory.submit_review
253     [BLOCK] sales.add_to_cart
254     [BLOCK] sales.process_return
255     [BLOCK] sales.checkout_cart
256     [ALLOW] sales.add_shipping_details (Success)
257     [BLOCK] inventory.bootstrap_new_store
258     [BLOCK] inventory.register_store
259     [BLOCK] sales.record_payment
260     [ALLOW] inventory.update_stock_level (Success)
261     [BLOCK] inventory.get_store_performance
262     [BLOCK] inventory.add_product
263     [BLOCK] customers.add_address
264     [BLOCK] customers.update_marketing_preferences
265     [ALLOW] sales.update_order_status (Success)
266     [BLOCK] audit.get_user_activity
267     [BLOCK] sales.request_return
268     [BLOCK] inventory.create_category
269     [BLOCK] inventory.create_discount
270     [ALLOW] auth.authenticate_user (Success)
271     [BLOCK] auth.register_customer
272
273 >>> TESTING USER: app_manager
274     [BLOCK] customers.create_wishlist
275     [BLOCK] customers.add_item_to_wishlist
276     [ALLOW] auth.register_employee (Success)
277     [BLOCK] inventory.submit_review
278     [BLOCK] sales.add_to_cart
279     [ALLOW] sales.process_return (Success)
280     [BLOCK] sales.checkout_cart
281     [ALLOW] sales.add_shipping_details (Success)
282     [BLOCK] inventory.bootstrap_new_store
283     [BLOCK] inventory.register_store
284     [BLOCK] sales.record_payment
285     [ALLOW] inventory.update_stock_level (Success)
286     [ALLOW] inventory.get_store_performance (Success)
287     [ALLOW] inventory.add_product (Logic Ran)
288     [BLOCK] customers.add_address

```

```

289      [BLOCK] customers.update_marketing_preferences
290      [ALLOW] sales.update_order_status (Success)
291      [BLOCK] audit.get_user_activity
292      [BLOCK] sales.request_return
293      [ALLOW] inventory.create_category (Success)
294      [ALLOW] inventory.create_discount (Success)
295      [ALLOW] auth.authenticate_user (Success)
296      [BLOCK] auth.register_customer
297
298  >>> TESTING USER: warehouse_bot
299      [BLOCK] customers.create_wishlist
300      [BLOCK] customers.add_item_to_wishlist
301      [BLOCK] auth.register_employee
302      [BLOCK] inventory.submit_review
303      [BLOCK] sales.add_to_cart
304      [BLOCK] sales.process_return
305      [BLOCK] sales.checkout_cart
306      [BLOCK] sales.add_shipping_details
307      [BLOCK] inventory.bootstrap_new_store
308      [BLOCK] inventory.register_store
309      [BLOCK] sales.record_payment
310      [ALLOW] inventory.update_stock_level (Success)
311      [BLOCK] inventory.get_store_performance
312      [BLOCK] inventory.add_product
313      [BLOCK] customers.add_address
314      [BLOCK] customers.update_marketing_preferences
315      [BLOCK] sales.update_order_status
316      [BLOCK] audit.get_user_activity
317      [BLOCK] sales.request_return
318      [BLOCK] inventory.create_category
319      [BLOCK] inventory.create_discount
320      [BLOCK] auth.authenticate_user
321      [BLOCK] auth.register_customer
322
323  >>> TESTING USER: sec_auditor
324      [BLOCK] customers.create_wishlist
325      [BLOCK] customers.add_item_to_wishlist
326      [BLOCK] auth.register_employee
327      [BLOCK] inventory.submit_review
328      [BLOCK] sales.add_to_cart
329      [BLOCK] sales.process_return
330      [BLOCK] sales.checkout_cart
331      [BLOCK] sales.add_shipping_details
332      [BLOCK] inventory.bootstrap_new_store
333      [BLOCK] inventory.register_store
334      [BLOCK] sales.record_payment
335      [BLOCK] inventory.update_stock_level
336      [BLOCK] inventory.get_store_performance
337      [BLOCK] inventory.add_product

```

```
338      [BLOCK]  customers.add_address
339      [BLOCK]  customers.update_marketing_preferences
340      [BLOCK]  sales.update_order_status
341      [ALLOW]  audit.get_user_activity  (Success)
342      [BLOCK]  sales.request_return
343      [BLOCK]  inventory.create_category
344      [BLOCK]  inventory.create_discount
345      [BLOCK]  auth.authenticate_user
346      [BLOCK]  auth.register_customer
347  =====
348      FULL  LIFECYCLE  TEST  COMPLETE
349  =====
350
```

C Code Appendix: Database Schema

Below is the core schema definition.

```
1  -- [INVENTORY SCHEMA]
2  -- Supports L01: Scalability for future marketplace expansion.
3  CREATE TABLE IF NOT EXISTS inventory.stores
4  (
5      store_id serial NOT NULL,
6      store_name character varying(100) NOT NULL,
7      owner_user_id integer NOT NULL REFERENCES auth.users(user_id),
8      created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
9      is_active boolean DEFAULT true,
10     CONSTRAINT stores_pkey PRIMARY KEY (store_id)
11 );
12
13 CREATE TABLE IF NOT EXISTS inventory.products
14 (
15     product_id serial NOT NULL,
16     product_name character varying(255) NOT NULL,
17     description text,
18     price numeric(10, 2),
19     stock_quantity integer DEFAULT 0,
20     store_id integer, -- Critical for Marketplace Scalability
21     CONSTRAINT products_pkey PRIMARY KEY (product_id)
22 );
```

D Code Appendix: Business Logic (Order Splitting)

The following function demonstrates the atomic handling of multi-vendor baskets.

```
1  CREATE OR REPLACE FUNCTION sales.checkout_cart(
2      p_customer_id INTEGER,
3      p_payment_method VARCHAR,
4      p_shipping_address_id INTEGER
5  ) RETURNS TEXT AS $$
6  DECLARE
7      v_cart_id INTEGER;
8      v_store_record RECORD;
9  BEGIN
10     -- [Logic Snippet] Loop through Distinct Stores in Cart
11     FOR v_store_record IN
12         SELECT DISTINCT p.store_id
13         FROM sales.cart_items ci
14         JOIN inventory.products p ON ci.product_id = p.product_id
15         WHERE ci.cart_id = v_cart_id
```

```

16         LOOP
17             -- 1. Create Order for this specific Store
18             INSERT INTO sales.orders (customer_id, store_id, ...)
19             VALUES (p_customer_id, v_store_record.store_id, ...)
20             RETURNING order_id INTO v_order_id;
21
22             -- 2. Move Items specific to this store...
23         END LOOP;
24         RETURN 'Checkout Complete';
25     END;
26 $$ LANGUAGE plpgsql SECURITY DEFINER;

```

E Full code copy from IM.sql

Note: test and setup scripts are in the TEST_SUIT folder in the zip file, so as this code

```

1 BEGIN;
2
3 CREATE EXTENSION IF NOT EXISTS pgcrypto;
4
5 -- =====
6 -- 1. Schema Definition
7 -- =====
8 CREATE SCHEMA IF NOT EXISTS audit;
9 CREATE SCHEMA IF NOT EXISTS auth;
10 CREATE SCHEMA IF NOT EXISTS customers;
11 CREATE SCHEMA IF NOT EXISTS hr;
12 CREATE SCHEMA IF NOT EXISTS inventory;
13 CREATE SCHEMA IF NOT EXISTS sales;
14
15 -- =====
16 -- 2. Table Definitions
17 -- =====
18
19 -- [AUDIT SCHEMA] (BCNF Compliant: Flat Log)
20 CREATE TABLE IF NOT EXISTS audit.audit_log
21 (
22     log_id serial NOT NULL,
23     user_id integer,
24     action_type character varying(50) NOT NULL,
25     table_name character varying(50) NOT NULL,
26     record_id integer,
27     old_value text,
28     new_value text,
29     ip_address inet,
30     "timestamp" timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
31     CONSTRAINT audit_log_pkey PRIMARY KEY (log_id)

```

```

32 );
33
34 -- [AUTH SCHEMA]
35 CREATE TABLE IF NOT EXISTS auth.roles
36 (
37     role_id serial NOT NULL,
38     role_name character varying(50) NOT NULL,
39     description text,
40     CONSTRAINT roles_pkey PRIMARY KEY (role_id),
41     CONSTRAINT roles_role_name_key UNIQUE (role_name)
42 );
43
44 CREATE TABLE IF NOT EXISTS auth.users
45 (
46     user_id serial NOT NULL,
47     email character varying(255) NOT NULL,
48     password_hash text NOT NULL,
49     first_name character varying(100),
50     last_name character varying(100),
51     phone_number character varying(20),
52     created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
53     is_active boolean DEFAULT true,
54     user_type character varying(20) NOT NULL,
55     CONSTRAINT users_pkey PRIMARY KEY (user_id),
56     CONSTRAINT users_email_key UNIQUE (email)
57 );
58
59 -- [CUSTOMERS SCHEMA]
60
61 -- BCNF FIX: Extracted City/Country to separate table.
62 -- Prevents transitive dependency (Postcode -> City, Country) inside Address table.
63 CREATE TABLE IF NOT EXISTS customers.cities
64 (
65     city_id serial NOT NULL,
66     city_name character varying(100) NOT NULL,
67     country_name character varying(100) NOT NULL,
68     CONSTRAINT cities_pkey PRIMARY KEY (city_id),
69     CONSTRAINT city_country_unique UNIQUE (city_name, country_name)
70 );
71
72 CREATE TABLE IF NOT EXISTS customers.customers
73 (
74     user_id integer NOT NULL,
75     marketing_opt_in boolean DEFAULT false,
76     CONSTRAINT customers_pkey PRIMARY KEY (user_id)
77 );
78
79 CREATE TABLE IF NOT EXISTS customers.addresses
80 (

```



```

81     address_id serial NOT NULL,
82     customer_id integer NOT NULL,
83     city_id integer NOT NULL, -- References new City table
84     address_line1 character varying(255) NOT NULL,
85     address_line2 character varying(255),
86     postcode character varying(20), -- Postcode remains as attribute of the specific
      ⇨ street address
87     CONSTRAINT addresses_pkey PRIMARY KEY (address_id)
88 );
89
90 CREATE TABLE IF NOT EXISTS customers.wishlist
91 (
92     wishlist_id serial NOT NULL,
93     customer_id integer NOT NULL,
94     wishlist_name character varying(100),
95     CONSTRAINT wishlist_pkey PRIMARY KEY (wishlist_id)
96 );
97
98 CREATE TABLE IF NOT EXISTS customers.wishlist_items
99 (
100     wishlist_item_id serial NOT NULL,
101     wishlist_id integer NOT NULL,
102     product_id integer NOT NULL,
103     added_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
104     CONSTRAINT wishlist_items_pkey PRIMARY KEY (wishlist_item_id)
105 );
106
107 -- [INVENTORY SCHEMA]
108 CREATE TABLE IF NOT EXISTS inventory.stores
109 (
110     store_id serial NOT NULL,
111     store_name character varying(100) NOT NULL,
112     owner_user_id integer NOT NULL REFERENCES auth.users(user_id),
113     created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
114     is_active boolean DEFAULT true,
115     CONSTRAINT stores_pkey PRIMARY KEY (store_id)
116 );
117
118 CREATE TABLE IF NOT EXISTS inventory.categories
119 (
120     category_id serial NOT NULL,
121     category_name character varying(100) NOT NULL,
122     description text,
123     CONSTRAINT categories_pkey PRIMARY KEY (category_id)
124 );
125
126 CREATE TABLE IF NOT EXISTS inventory.products
127 (
128     product_id serial NOT NULL,

```

```

129     product_name character varying(255) NOT NULL,
130     description text,
131     price numeric(10, 2),
132     stock_quantity integer DEFAULT 0,
133     store_id integer NOT NULL,
134     CONSTRAINT products_pkey PRIMARY KEY (product_id)
135 );
136
137 CREATE TABLE IF NOT EXISTS inventory.product_categories
138 (
139     product_id integer NOT NULL,
140     category_id integer NOT NULL,
141     CONSTRAINT product_categories_pkey PRIMARY KEY (product_id, category_id)
142 );
143
144 CREATE TABLE IF NOT EXISTS inventory.discounts
145 (
146     discount_id serial NOT NULL,
147     code character varying(50) NOT NULL,
148     discount_percent numeric(5, 2),
149     valid_from timestamp without time zone,
150     valid_to timestamp without time zone,
151     CONSTRAINT discounts_pkey PRIMARY KEY (discount_id),
152     CONSTRAINT discounts_code_key UNIQUE (code)
153 );
154
155 CREATE TABLE IF NOT EXISTS inventory.reviews
156 (
157     review_id serial NOT NULL,
158     customer_id integer NOT NULL,
159     product_id integer NOT NULL,
160     rating integer,
161     comment text,
162     review_date timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
163     CONSTRAINT reviews_pkey PRIMARY KEY (review_id)
164 );
165
166 -- [HR SCHEMA]
167 CREATE TABLE IF NOT EXISTS hr.employees
168 (
169     user_id integer NOT NULL,
170     role_id integer NOT NULL,
171     store_id integer,
172     department character varying(100),
173     CONSTRAINT employees_pkey PRIMARY KEY (user_id)
174 );
175
176 -- [SALES SCHEMA]
177

```

```

178 -- BCNF FIX: Removed 'total_amount' (Derived Data).
179 -- BCNF FIX: Removed 'shipping_address_id' (Transitive Dependency: Order -> Address ->
    ↳ Customer).
180 -- The link to address is now exclusively in the 'shipping' table.
181 CREATE TABLE IF NOT EXISTS sales.orders
182 (
183     order_id serial NOT NULL,
184     customer_id integer NOT NULL,
185     store_id integer NOT NULL,
186     order_date timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
187     status character varying(50) DEFAULT 'Pending',
188     CONSTRAINT orders_pkey PRIMARY KEY (order_id)
189 );
190
191 CREATE TABLE IF NOT EXISTS sales.order_items
192 (
193     order_item_id serial NOT NULL,
194     order_id integer NOT NULL,
195     product_id integer NOT NULL,
196     quantity integer NOT NULL,
197     unit_price numeric(10, 2) NOT NULL,
198     CONSTRAINT order_items_pkey PRIMARY KEY (order_item_id)
199 );
200
201 CREATE TABLE IF NOT EXISTS sales.shopping_carts
202 (
203     cart_id serial NOT NULL,
204     customer_id integer,
205     created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
206     CONSTRAINT shopping_carts_pkey PRIMARY KEY (cart_id),
207     CONSTRAINT shopping_carts_customer_key UNIQUE (customer_id)
208 );
209
210 CREATE TABLE IF NOT EXISTS sales.cart_items
211 (
212     cart_item_id serial NOT NULL,
213     cart_id integer NOT NULL,
214     product_id integer NOT NULL,
215     quantity integer DEFAULT 1,
216     CONSTRAINT cart_items_pkey PRIMARY KEY (cart_item_id)
217 );
218
219 CREATE TABLE IF NOT EXISTS sales.payments
220 (
221     payment_id serial NOT NULL,
222     order_id integer NOT NULL,
223     payment_date timestamp without time zone DEFAULT CURRENT_TIMESTAMP,

```

```

224     amount numeric(10, 2), -- This remains as it records the ACTUAL transaction
      ↳ amount, which may differ from calculated order total due to refunds/partial
      ↳ payments.
225     payment_method character varying(50),
226     status character varying(50),
227     CONSTRAINT payments_pkey PRIMARY KEY (payment_id)
228 );
229
230 CREATE TABLE IF NOT EXISTS sales.returns
231 (
232     return_id serial NOT NULL,
233     order_id integer NOT NULL,
234     employee_id integer,
235     reason text,
236     status character varying(50) DEFAULT 'Requested',
237     created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
238     CONSTRAINT returns_pkey PRIMARY KEY (return_id)
239 );
240
241 CREATE TABLE IF NOT EXISTS sales.return_items
242 (
243     return_item_id serial NOT NULL,
244     return_id integer NOT NULL,
245     product_id integer NOT NULL,
246     quantity integer NOT NULL,
247     CONSTRAINT return_items_pkey PRIMARY KEY (return_item_id)
248 );
249
250 -- BCNF FIX: Shipping details now hold the address link.
251 -- This decouples the Order entity from the Address entity.
252 CREATE TABLE IF NOT EXISTS sales.shipping
253 (
254     shipping_id serial NOT NULL,
255     order_id integer NOT NULL,
256     address_id integer NOT NULL REFERENCES customers.addresses(address_id), -- Moved
      ↳ from Orders
257     shipping_method character varying(100),
258     tracking_number character varying(100),
259     status character varying(50),
260     shipped_date timestamp without time zone,
261     CONSTRAINT shipping_pkey PRIMARY KEY (shipping_id)
262 );
263
264 -- =====
265 -- 3. Foreign Key Constraints
266 -- =====
267
268 -- Audit & Auth
269 ALTER TABLE IF EXISTS audit.audit_log

```

```

270     ADD CONSTRAINT audit_log_user_fkey FOREIGN KEY (user_id) REFERENCES auth.users
      ↪ (user_id) ON DELETE SET NULL;
271
272 -- Customer & Geo
273 ALTER TABLE IF EXISTS customers.customers
274     ADD CONSTRAINT customers_users_fkey FOREIGN KEY (user_id) REFERENCES auth.users
      ↪ (user_id) ON DELETE CASCADE;
275
276 ALTER TABLE IF EXISTS customers.addresses
277     ADD CONSTRAINT addresses_customer_fkey FOREIGN KEY (customer_id) REFERENCES
      ↪ customers.customers (user_id) ON DELETE CASCADE;
278
279 ALTER TABLE IF EXISTS customers.addresses
280     ADD CONSTRAINT addresses_city_fkey FOREIGN KEY (city_id) REFERENCES
      ↪ customers.cities (city_id) ON DELETE RESTRICT;
281
282 ALTER TABLE IF EXISTS customers.wishlist
283     ADD CONSTRAINT wishlist_customer_fkey FOREIGN KEY (customer_id) REFERENCES
      ↪ customers.customers (user_id);
284
285 ALTER TABLE IF EXISTS customers.wishlist_items
286     ADD CONSTRAINT wishlist_items_list_fkey FOREIGN KEY (wishlist_id) REFERENCES
      ↪ customers.wishlist (wishlist_id);
287 ALTER TABLE IF EXISTS customers.wishlist_items
288     ADD CONSTRAINT wishlist_items_prod_fkey FOREIGN KEY (product_id) REFERENCES
      ↪ inventory.products (product_id);
289
290 -- HR
291 ALTER TABLE IF EXISTS hr.employees
292     ADD CONSTRAINT employees_users_fkey FOREIGN KEY (user_id) REFERENCES auth.users
      ↪ (user_id) ON DELETE CASCADE;
293 ALTER TABLE IF EXISTS hr.employees
294     ADD CONSTRAINT employees_role_id_fkey FOREIGN KEY (role_id) REFERENCES auth.roles
      ↪ (role_id);
295 ALTER TABLE IF EXISTS hr.employees
296     ADD CONSTRAINT employees_store_fkey FOREIGN KEY (store_id) REFERENCES
      ↪ inventory.stores (store_id) ON DELETE SET NULL;
297
298 -- Inventory
299 ALTER TABLE IF EXISTS inventory.products
300     ADD CONSTRAINT products_store_fkey FOREIGN KEY (store_id) REFERENCES
      ↪ inventory.stores (store_id);
301 ALTER TABLE IF EXISTS inventory.product_categories
302     ADD CONSTRAINT fk_prod_cat_cat FOREIGN KEY (category_id) REFERENCES
      ↪ inventory.categories (category_id);
303 ALTER TABLE IF EXISTS inventory.product_categories
304     ADD CONSTRAINT fk_prod_cat_prod FOREIGN KEY (product_id) REFERENCES
      ↪ inventory.products (product_id);
305 ALTER TABLE IF EXISTS inventory.reviews

```

```

306     ADD CONSTRAINT reviews_customer_fkey FOREIGN KEY (customer_id) REFERENCES
        ↳ customers.customers (user_id);
307 ALTER TABLE IF EXISTS inventory.reviews
308     ADD CONSTRAINT reviews_product_fkey FOREIGN KEY (product_id) REFERENCES
        ↳ inventory.products (product_id);
309
310 -- Sales
311 ALTER TABLE IF EXISTS sales.orders
312     ADD CONSTRAINT orders_customer_fkey FOREIGN KEY (customer_id) REFERENCES
        ↳ customers.customers (user_id);
313 ALTER TABLE IF EXISTS sales.orders
314     ADD CONSTRAINT orders_store_fkey FOREIGN KEY (store_id) REFERENCES
        ↳ inventory.stores (store_id);
315
316 ALTER TABLE IF EXISTS sales.order_items
317     ADD CONSTRAINT order_items_order_fkey FOREIGN KEY (order_id) REFERENCES
        ↳ sales.orders (order_id);
318 ALTER TABLE IF EXISTS sales.order_items
319     ADD CONSTRAINT order_items_product_fkey FOREIGN KEY (product_id) REFERENCES
        ↳ inventory.products (product_id);
320
321 ALTER TABLE IF EXISTS sales.shopping_carts
322     ADD CONSTRAINT shopping_carts_customer_fkey FOREIGN KEY (customer_id) REFERENCES
        ↳ customers.customers (user_id) ON DELETE CASCADE;
323
324 ALTER TABLE IF EXISTS sales.cart_items
325     ADD CONSTRAINT cart_items_cart_fkey FOREIGN KEY (cart_id) REFERENCES
        ↳ sales.shopping_carts (cart_id);
326 ALTER TABLE IF EXISTS sales.cart_items
327     ADD CONSTRAINT cart_items_product_fkey FOREIGN KEY (product_id) REFERENCES
        ↳ inventory.products (product_id);
328
329 ALTER TABLE IF EXISTS sales.payments
330     ADD CONSTRAINT payments_order_fkey FOREIGN KEY (order_id) REFERENCES sales.orders
        ↳ (order_id);
331
332 ALTER TABLE IF EXISTS sales.returns
333     ADD CONSTRAINT returns_employee_fkey FOREIGN KEY (employee_id) REFERENCES
        ↳ hr.employees (user_id);
334 ALTER TABLE IF EXISTS sales.returns
335     ADD CONSTRAINT returns_order_fkey FOREIGN KEY (order_id) REFERENCES sales.orders
        ↳ (order_id);
336
337 ALTER TABLE IF EXISTS sales.return_items
338     ADD CONSTRAINT return_items_return_fkey FOREIGN KEY (return_id) REFERENCES
        ↳ sales.returns (return_id);
339 ALTER TABLE IF EXISTS sales.return_items
340     ADD CONSTRAINT return_items_product_fkey FOREIGN KEY (product_id) REFERENCES
        ↳ inventory.products (product_id);

```

```

341
342 ALTER TABLE IF EXISTS sales.shipping
343     ADD CONSTRAINT shipping_order_fkey FOREIGN KEY (order_id) REFERENCES sales.orders
      ↪ (order_id);
344
345 -- =====
346 -- 4. Views (Recovering Usability)
347 -- =====
348
349 -- VIEW: Order Summary
350 -- Replaces the removed 'total_amount' column by calculating it on demand.
351 CREATE OR REPLACE VIEW sales.v_order_summary AS
352 SELECT
353     o.order_id,
354     o.customer_id,
355     o.store_id,
356     o.order_date,
357     o.status,
358     COALESCE(SUM(oi.quantity * oi.unit_price), 0.00) AS total_amount,
359     s.address_id,
360     s.shipping_method,
361     s.status AS shipping_status
362 FROM sales.orders o
363 LEFT JOIN sales.order_items oi ON o.order_id = oi.order_id
364 LEFT JOIN sales.shipping s ON o.order_id = s.order_id
365 GROUP BY o.order_id, s.address_id, s.shipping_method, s.status;
366
367 -- VIEW: Full Addresses
368 -- Re-assembles the fragmented BCNF address structure for easy UI consumption.
369 CREATE OR REPLACE VIEW customers.v_full_addresses AS
370 SELECT
371     a.address_id,
372     a.customer_id,
373     a.address_line1,
374     a.address_line2,
375     c.city_name,
376     a.postcode,
377     c.country_name
378 FROM customers.addresses a
379 JOIN customers.cities c ON a.city_id = c.city_id;
380
381 -- =====
382 -- 5. Indexes
383 -- =====
384 CREATE INDEX IF NOT EXISTS idx_customers_user_id ON customers.customers(user_id);
385 CREATE INDEX IF NOT EXISTS idx_employees_user_id ON hr.employees(user_id);
386 CREATE INDEX IF NOT EXISTS idx_products_store_id ON inventory.products(store_id);
387 CREATE INDEX IF NOT EXISTS idx_orders_store_id ON sales.orders(store_id);

```

```

388 CREATE INDEX IF NOT EXISTS idx_shopping_carts_customer ON
    ↪ sales.shopping_carts(customer_id);
389 CREATE INDEX IF NOT EXISTS idx_addresses_city ON customers.addresses(city_id);
390
391 COMMIT;
392
393 -- =====
394 -- 1. AUTHENTICATION & REGISTRATION
395 -- =====
396
397 -- 1.1 Register Customer
398 CREATE OR REPLACE FUNCTION auth.register_customer(
399     p_email VARCHAR,
400     p_password VARCHAR,
401     p_first_name VARCHAR,
402     p_last_name VARCHAR
403 ) RETURNS INTEGER AS $$
404 DECLARE
405     v_user_id INTEGER;
406 BEGIN
407     -- Check if email exists
408     IF EXISTS (SELECT 1 FROM auth.users WHERE email = p_email) THEN
409         RAISE EXCEPTION 'Email already registered';
410     END IF;
411
412     -- Insert into auth.users with Hashed Password
413     INSERT INTO auth.users (email, password_hash, first_name, last_name, user_type)
414     VALUES (p_email, crypt(p_password, gen_salt('bf')), p_first_name, p_last_name,
    ↪ 'Customer')
415     RETURNING user_id INTO v_user_id;
416
417     -- Insert into customers.customers
418     INSERT INTO customers.customers (user_id, marketing_opt_in)
419     VALUES (v_user_id, false);
420
421     RETURN v_user_id;
422 END;
423 $$ LANGUAGE plpgsql SECURITY DEFINER;
424
425 -- 1.2 Register Employee
426 CREATE OR REPLACE FUNCTION auth.register_employee(
427     p_email VARCHAR,
428     p_password VARCHAR,
429     p_first_name VARCHAR,
430     p_last_name VARCHAR,
431     p_role_name VARCHAR,
432     p_department VARCHAR,
433     p_store_id INTEGER
434 ) RETURNS INTEGER AS $$

```



```

435 DECLARE
436     v_user_id INTEGER;
437     v_role_id INTEGER;
438 BEGIN
439     -- Get Role ID
440     SELECT role_id INTO v_role_id FROM auth.roles WHERE role_name = p_role_name;
441     IF v_role_id IS NULL THEN
442         RAISE EXCEPTION 'Invalid Role Name';
443     END IF;
444
445     -- Insert into auth.users
446     INSERT INTO auth.users (email, password_hash, first_name, last_name, user_type)
447     VALUES (p_email, crypt(p_password, gen_salt('bf')), p_first_name, p_last_name,
448             ⇨ 'Employee')
449     RETURNING user_id INTO v_user_id;
450
451     -- Insert into hr.employees
452     INSERT INTO hr.employees (user_id, role_id, store_id, department)
453     VALUES (v_user_id, v_role_id, p_store_id, p_department);
454
455     RETURN v_user_id;
456 END;
457
458 $$ LANGUAGE plpgsql SECURITY DEFINER;
459
460 -- 1.3 Authenticate User
461 CREATE OR REPLACE FUNCTION auth.authenticate_user(
462     p_email VARCHAR,
463     p_input_password VARCHAR
464 ) RETURNS INTEGER AS $$
465 DECLARE
466     v_user_id INTEGER;
467     v_password_hash TEXT;
468 BEGIN
469     SELECT user_id, password_hash INTO v_user_id, v_password_hash
470     FROM auth.users
471     WHERE email = p_email AND is_active = true;
472
473     IF v_user_id IS NOT NULL AND v_password_hash = crypt(p_input_password,
474             ⇨ v_password_hash) THEN
475         RETURN v_user_id; -- Authentication Successful
476     ELSE
477         RETURN NULL; -- Authentication Failed
478     END IF;
479 END;
480
481 $$ LANGUAGE plpgsql SECURITY DEFINER;
482
483 -- =====
484 -- 2. CUSTOMER MANAGEMENT (BCNF UPDATES)
485 -- =====

```

```

482
483 -- 2.1 Add Address
484 CREATE OR REPLACE FUNCTION customers.add_address(
485     p_user_id INTEGER,
486     p_line1 VARCHAR,
487     p_city VARCHAR,
488     p_postcode VARCHAR,
489     p_country VARCHAR
490 ) RETURNS VOID AS $$
491 DECLARE
492     v_city_id INTEGER;
493 BEGIN
494     -- 1. Ensure City/Country exists in normalised table
495     INSERT INTO customers.cities (city_name, country_name)
496     VALUES (p_city, p_country)
497     ON CONFLICT (city_name, country_name) DO NOTHING;
498
499     -- 2. Retrieve the City ID
500     SELECT city_id INTO v_city_id
501     FROM customers.cities
502     WHERE city_name = p_city AND country_name = p_country;
503
504     -- 3. Insert Address using ID instead of text
505     INSERT INTO customers.addresses (customer_id, city_id, address_line1, postcode)
506     VALUES (p_user_id, v_city_id, p_line1, p_postcode);
507 END;
508 $$ LANGUAGE plpgsql;
509
510 -- 2.2 Update Marketing Preferences
511 CREATE OR REPLACE FUNCTION customers.update_marketing_preferences(
512     p_user_id INTEGER,
513     p_opt_in BOOLEAN
514 ) RETURNS VOID AS $$
515 BEGIN
516     UPDATE customers.customers
517     SET marketing_opt_in = p_opt_in
518     WHERE user_id = p_user_id;
519 END;
520 $$ LANGUAGE plpgsql;
521
522 -- 2.3 Create Wishlist
523 CREATE OR REPLACE FUNCTION customers.create_wishlist(
524     p_user_id INTEGER,
525     p_wishlist_name VARCHAR
526 ) RETURNS INTEGER AS $$
527 DECLARE
528     v_wishlist_id INTEGER;
529 BEGIN
530     INSERT INTO customers.wishlist (customer_id, wishlist_name)

```

```

531     VALUES (p_user_id, p_wishlist_name)
532     RETURNING wishlist_id INTO v_wishlist_id;
533     RETURN v_wishlist_id;
534 END;
535 $$ LANGUAGE plpgsql;
536
537 -- 2.4 Add Item to Wishlist
538 CREATE OR REPLACE FUNCTION customers.add_item_to_wishlist(
539     p_wishlist_id INTEGER,
540     p_product_id INTEGER
541 ) RETURNS VOID AS $$
542 BEGIN
543     INSERT INTO customers.wishlist_items (wishlist_id, product_id)
544     VALUES (p_wishlist_id, p_product_id);
545 END;
546 $$ LANGUAGE plpgsql;
547
548 -- =====
549 -- 3. INVENTORY MANAGEMENT
550 -- =====
551
552 -- 3.1 Create Category
553 CREATE OR REPLACE FUNCTION inventory.create_category(
554     p_category_name VARCHAR,
555     p_description TEXT
556 ) RETURNS INTEGER AS $$
557 DECLARE
558     v_cat_id INTEGER;
559 BEGIN
560     INSERT INTO inventory.categories (category_name, description)
561     VALUES (p_category_name, p_description)
562     RETURNING category_id INTO v_cat_id;
563     RETURN v_cat_id;
564 END;
565 $$ LANGUAGE plpgsql;
566
567 -- 3.2 Add Product
568 CREATE OR REPLACE FUNCTION inventory.add_product(
569     p_store_id INTEGER,
570     p_product_name VARCHAR,
571     p_price NUMERIC,
572     p_stock_count INTEGER
573 ) RETURNS INTEGER AS $$
574 DECLARE
575     v_prod_id INTEGER;
576 BEGIN
577     INSERT INTO inventory.products (store_id, product_name, price, stock_quantity)
578     VALUES (p_store_id, p_product_name, p_price, p_stock_count)
579     RETURNING product_id INTO v_prod_id;

```

```

580     RETURN v_prod_id;
581 END;
582 $$ LANGUAGE plpgsql;
583
584 -- 3.3 Update Stock Level
585 CREATE OR REPLACE FUNCTION inventory.update_stock_level(
586     p_product_id INTEGER,
587     p_quantity_change INTEGER
588 ) RETURNS VOID AS $$
589 BEGIN
590     UPDATE inventory.products
591     SET stock_quantity = stock_quantity + p_quantity_change
592     WHERE product_id = p_product_id;
593
594     -- Constraint check handled by Trigger 3.5
595 END;
596 $$ LANGUAGE plpgsql;
597
598 -- 3.4 Create Discount
599 CREATE OR REPLACE FUNCTION inventory.create_discount(
600     p_code VARCHAR,
601     p_percent NUMERIC,
602     p_valid_from TIMESTAMP WITH TIME ZONE,
603     p_valid_to TIMESTAMP WITH TIME ZONE
604 ) RETURNS INTEGER AS $$
605 DECLARE
606     v_id INTEGER;
607 BEGIN
608     IF p_percent <= 0 OR p_percent > 100 THEN
609         RAISE EXCEPTION 'Discount must be between 1%% and 100%%';
610     END IF;
611
612     INSERT INTO inventory.discounts (code, discount_percent, valid_from, valid_to)
613     VALUES (p_code, p_percent, p_valid_from::timestamp, p_valid_to::timestamp)
614     RETURNING discount_id INTO v_id;
615     RETURN v_id;
616 END;
617 $$ LANGUAGE plpgsql;
618
619 -- 3.5 Prevent Negative Stock
620 CREATE OR REPLACE FUNCTION inventory.prevent_negative_stock()
621 RETURNS TRIGGER AS $$
622 BEGIN
623     IF NEW.stock_quantity < 0 THEN
624         RAISE EXCEPTION 'Stock quantity cannot be negative. Product ID: %',
625             ↪ NEW.product_id;
626     END IF;
627     RETURN NEW;
628 END;

```

```

628 $$ LANGUAGE plpgsql;
629
630 CREATE TRIGGER enforce_stock_levels
631 BEFORE INSERT OR UPDATE ON inventory.products
632 FOR EACH ROW EXECUTE FUNCTION inventory.prevent_negative_stock();
633
634 -- =====
635 -- 4. STORE MANAGEMENT & BOOTSTRAP
636 -- =====
637
638 -- 4.1 Register Store
639 CREATE OR REPLACE FUNCTION inventory.register_store(
640     p_store_name VARCHAR,
641     p_owner_user_id INTEGER
642 ) RETURNS INTEGER AS $$
643 DECLARE
644     v_store_id INTEGER;
645 BEGIN
646     INSERT INTO inventory.stores (store_name, owner_user_id)
647     VALUES (p_store_name, p_owner_user_id)
648     RETURNING store_id INTO v_store_id;
649     RETURN v_store_id;
650 END;
651 $$ LANGUAGE plpgsql;
652
653 -- 4.2 Get Store Performance
654 CREATE OR REPLACE FUNCTION inventory.get_store_performance(
655     p_store_id INTEGER
656 ) RETURNS NUMERIC AS $$
657 DECLARE
658     v_total_revenue NUMERIC;
659 BEGIN
660     SELECT COALESCE(SUM(total_amount), 0) INTO v_total_revenue
661     FROM sales.v_order_summary
662     WHERE store_id = p_store_id AND status != 'Cancelled';
663
664     RETURN v_total_revenue;
665 END;
666 $$ LANGUAGE plpgsql;
667
668 -- 1.3 Bootstrap First Store
669 CREATE OR REPLACE FUNCTION inventory.bootstrap_new_store(
670     p_email VARCHAR,
671     p_password VARCHAR,
672     p_first_name VARCHAR,
673     p_last_name VARCHAR,
674     p_store_name VARCHAR
675 ) RETURNS TEXT AS $$
676 DECLARE

```

```

677     v_manager_id INTEGER;
678     v_store_id INTEGER;
679     v_role_id INTEGER;
680 BEGIN
681     -- 1. Create User
682     INSERT INTO auth.users (email, password_hash, first_name, last_name, user_type)
683     VALUES (p_email, crypt(p_password, gen_salt('bf')), p_first_name, p_last_name,
684             ↪ 'Employee')
685     RETURNING user_id INTO v_manager_id;
686
687     -- 2. Create Store
688     INSERT INTO inventory.stores (store_name, owner_user_id)
689     VALUES (p_store_name, v_manager_id)
690     RETURNING store_id INTO v_store_id;
691
692     -- 3. Ensure Role Exists
693     SELECT role_id INTO v_role_id FROM auth.roles WHERE role_name = 'Store Manager';
694     IF v_role_id IS NULL THEN
695         INSERT INTO auth.roles (role_name, description)
696         VALUES ('Store Manager', 'Oversees store inventory and staff')
697         RETURNING role_id INTO v_role_id;
698     END IF;
699
700     -- 4. Create Employee Record
701     INSERT INTO hr.employees (user_id, role_id, store_id, department)
702     VALUES (v_manager_id, v_role_id, v_store_id, 'Management');
703
704     RETURN 'Success: Created Store "' || p_store_name || '" with Manager ID ' ||
705     ↪ v_manager_id;
706 END;
707 $$$ LANGUAGE plpgsql SECURITY DEFINER;
708
709 -- =====
710 -- 5. SALES & CHECKOUT
711 -- =====
712
713 -- 5.1 Add to Cart
714 CREATE OR REPLACE FUNCTION sales.add_to_cart(
715     p_customer_id INTEGER,
716     p_product_id INTEGER,
717     p_quantity INTEGER
718 ) RETURNS VOID AS $$$
719 DECLARE
720     v_cart_id INTEGER;
721 BEGIN
722     INSERT INTO sales.shopping_carts (customer_id)
723     VALUES (p_customer_id)
724     ON CONFLICT (customer_id) DO NOTHING;
725
726

```

```

724     SELECT cart_id INTO v_cart_id FROM sales.shopping_carts WHERE customer_id =
       ↪ p_customer_id;
725
726     IF EXISTS (SELECT 1 FROM sales.cart_items WHERE cart_id = v_cart_id AND product_id
       ↪ = p_product_id) THEN
727         UPDATE sales.cart_items
728         SET quantity = quantity + p_quantity
729         WHERE cart_id = v_cart_id AND product_id = p_product_id;
730     ELSE
731         INSERT INTO sales.cart_items (cart_id, product_id, quantity)
732         VALUES (v_cart_id, p_product_id, p_quantity);
733     END IF;
734 END;
735 $$ LANGUAGE plpgsql;
736
737 -- 5.2 Checkout Cart
738 CREATE OR REPLACE FUNCTION sales.checkout_cart(
739     p_customer_id INTEGER,
740     p_payment_method VARCHAR,
741     p_shipping_address_id INTEGER
742 ) RETURNS TEXT AS $$
743 DECLARE
744     v_cart_id INTEGER;
745     v_store_record RECORD;
746     v_item_record RECORD;
747     v_order_id INTEGER;
748     v_current_order_total NUMERIC;
749 BEGIN
750     -- Get Cart ID
751     SELECT cart_id INTO v_cart_id FROM sales.shopping_carts WHERE customer_id =
       ↪ p_customer_id;
752     IF v_cart_id IS NULL THEN RAISE EXCEPTION 'Cart is empty or does not exist'; END
       ↪ IF;
753
754     -- Loop through Distinct Stores in Cart
755     FOR v_store_record IN
756         SELECT DISTINCT p.store_id
757         FROM sales.cart_items ci
758         JOIN inventory.products p ON ci.product_id = p.product_id
759         WHERE ci.cart_id = v_cart_id
760     LOOP
761         v_current_order_total := 0;
762
763         -- 1. Create Order (Minimal info)
764         INSERT INTO sales.orders (customer_id, store_id, status)
765         VALUES (p_customer_id, v_store_record.store_id, 'Pending')
766         RETURNING order_id INTO v_order_id;
767
768         -- 2. Create Shipping Record (Links Address here, NOT in Orders table)

```

```

769     INSERT INTO sales.shipping (order_id, address_id, status)
770     VALUES (v_order_id, p_shipping_address_id, 'Pending');
771
772     -- 3. Move Items
773     FOR v_item_record IN
774         SELECT ci.product_id, ci.quantity, p.price
775         FROM sales.cart_items ci
776         JOIN inventory.products p ON ci.product_id = p.product_id
777         WHERE ci.cart_id = v_cart_id AND p.store_id = v_store_record.store_id
778     LOOP
779         -- Check Stock
780         PERFORM inventory.update_stock_level(v_item_record.product_id,
781             ↪ -v_item_record.quantity);
782
783         -- Insert Order Item
784         INSERT INTO sales.order_items (order_id, product_id, quantity, unit_price)
785         VALUES (v_order_id, v_item_record.product_id, v_item_record.quantity,
786             ↪ v_item_record.price);
787
788         -- Calculate Total (For Payment record only, not for Order table)
789         v_current_order_total := v_current_order_total + (v_item_record.price *
790             ↪ v_item_record.quantity);
791     END LOOP;
792
793     -- 4. Record Payment (Stores actual paid amount)
794     PERFORM sales.record_payment(v_order_id, v_current_order_total,
795         ↪ p_payment_method, 'Completed');
796
797     END LOOP;
798
799     -- 5. Clear Cart
800     DELETE FROM sales.cart_items WHERE cart_id = v_cart_id;
801
802     RETURN 'Checkout Complete: Orders created per store.';
803 END;
804
805 $$ LANGUAGE plpgsql;
806
807 -- 5.3 Record Payment
808 CREATE OR REPLACE FUNCTION sales.record_payment(
809     p_order_id INTEGER,
810     p_amount NUMERIC,
811     p_method VARCHAR,
812     p_status VARCHAR
813 ) RETURNS VOID AS $$
814 BEGIN
815     INSERT INTO sales.payments (order_id, amount, payment_method, status)
816     VALUES (p_order_id, p_amount, p_method, p_status);
817 END;
818
819 $$ LANGUAGE plpgsql;

```



```

814
815 -- 6.1 Update Order Status
816 CREATE OR REPLACE FUNCTION sales.update_order_status(
817     p_order_id INTEGER,
818     p_new_status VARCHAR
819 ) RETURNS VOID AS $$
820 BEGIN
821     UPDATE sales.orders
822     SET status = p_new_status
823     WHERE order_id = p_order_id;
824 END;
825 $$ LANGUAGE plpgsql;
826
827 -- 6.2 Add Shipping Details
828 CREATE OR REPLACE FUNCTION sales.add_shipping_details(
829     p_order_id INTEGER,
830     p_carrier VARCHAR,
831     p_tracking VARCHAR
832 ) RETURNS VOID AS $$
833 BEGIN
834     -- Update existing record (created at checkout) with tracking info
835     UPDATE sales.shipping
836     SET shipping_method = p_carrier,
837         tracking_number = p_tracking,
838         status = 'Shipped',
839         shipped_date = NOW()
840     WHERE order_id = p_order_id;
841
842     -- Auto-update order status
843     PERFORM sales.update_order_status(p_order_id, 'Shipped');
844 END;
845 $$ LANGUAGE plpgsql;
846
847 -- =====
848 -- 7. REVIEWS & RETURNS
849 -- =====
850
851 -- 7.1 Submit Review
852 CREATE OR REPLACE FUNCTION inventory.submit_review(
853     p_customer_id INTEGER,
854     p_product_id INTEGER,
855     p_rating INTEGER,
856     p_comment TEXT
857 ) RETURNS VOID AS $$
858 BEGIN
859     INSERT INTO inventory.reviews (customer_id, product_id, rating, comment)
860     VALUES (p_customer_id, p_product_id, p_rating, p_comment);
861 END;
862 $$ LANGUAGE plpgsql;

```

```

863
864 -- 7.2 Request Return
865 CREATE OR REPLACE FUNCTION sales.request_return(
866     p_order_id INTEGER,
867     p_product_id INTEGER,
868     p_reason TEXT
869 ) RETURNS INTEGER AS $$
870 DECLARE
871     v_return_id INTEGER;
872 BEGIN
873     INSERT INTO sales.returns (order_id, reason, status)
874     VALUES (p_order_id, p_reason, 'Requested')
875     RETURNING return_id INTO v_return_id;
876
877     INSERT INTO sales.return_items (return_id, product_id, quantity)
878     VALUES (v_return_id, p_product_id, 1);
879
880     RETURN v_return_id;
881 END;
882 $$ LANGUAGE plpgsql;
883
884 -- 7.3 Process Return
885 CREATE OR REPLACE FUNCTION sales.process_return(
886     p_return_id INTEGER,
887     p_employee_id INTEGER,
888     p_status VARCHAR
889 ) RETURNS VOID AS $$
890 BEGIN
891     UPDATE sales.returns
892     SET status = p_status, employee_id = p_employee_id
893     WHERE return_id = p_return_id;
894 END;
895 $$ LANGUAGE plpgsql;
896
897 -- =====
898 -- 8. AUDIT LOGGING
899 -- =====
900
901 -- 8.1 Audit Trigger Function
902 CREATE OR REPLACE FUNCTION audit.log_event()
903 RETURNS TRIGGER AS $$
904 DECLARE
905     v_user_id INTEGER;
906     v_old_val TEXT;
907     v_new_val TEXT;
908 BEGIN
909     BEGIN
910         v_user_id := current_setting('app.current_user_id')::INTEGER;
911     EXCEPTION WHEN OTHERS THEN

```

```

912         v_user_id := NULL;
913     END;
914
915     IF (TG_OP = 'DELETE') THEN
916         v_old_val := row_to_json(OLD)::TEXT;
917         v_new_val := NULL;
918     ELSIF (TG_OP = 'UPDATE') THEN
919         v_old_val := row_to_json(OLD)::TEXT;
920         v_new_val := row_to_json(NEW)::TEXT;
921     ELSE
922         v_old_val := NULL;
923         v_new_val := row_to_json(NEW)::TEXT;
924     END IF;
925
926     INSERT INTO audit.audit_log (user_id, action_type, table_name, old_value,
927         ↪ new_value, ip_address)
928     VALUES (v_user_id, TG_OP, TG_TABLE_NAME, v_old_val, v_new_val,
929         ↪ inet_client_addr());
930     RETURN NEW;
931 END;
932 $$ LANGUAGE plpgsql SECURITY DEFINER;
933
934 -- 8.2 Get User Activity
935 CREATE OR REPLACE FUNCTION audit.get_user_activity(p_user_id INTEGER)
936 RETURNS TABLE(action VARCHAR, tbl VARCHAR, time_val TIMESTAMP) AS $$
937 BEGIN
938     RETURN QUERY
939     SELECT action_type, table_name, timestamp
940     FROM audit.audit_log
941     WHERE user_id = p_user_id
942     ORDER BY timestamp DESC;
943 END;
944 $$ LANGUAGE plpgsql;
945
946 -- Apply Triggers
947 CREATE TRIGGER audit_users_trigger AFTER INSERT OR UPDATE OR DELETE ON auth.users FOR
948     ↪ EACH ROW EXECUTE FUNCTION audit.log_event();
949 CREATE TRIGGER audit_orders_trigger AFTER INSERT OR UPDATE OR DELETE ON sales.orders
950     ↪ FOR EACH ROW EXECUTE FUNCTION audit.log_event();
951 CREATE TRIGGER audit_employees_trigger AFTER INSERT OR UPDATE OR DELETE ON
952     ↪ hr.employees FOR EACH ROW EXECUTE FUNCTION audit.log_event();
953 CREATE TRIGGER audit_payments_trigger AFTER INSERT OR UPDATE OR DELETE ON
954     ↪ sales.payments FOR EACH ROW EXECUTE FUNCTION audit.log_event();
955 CREATE TRIGGER audit_products_trigger AFTER INSERT OR UPDATE OR DELETE ON
956     ↪ inventory.products FOR EACH ROW EXECUTE FUNCTION audit.log_event();
957 CREATE TRIGGER audit_addresses_trigger AFTER INSERT OR UPDATE OR DELETE ON
958     ↪ customers.addresses FOR EACH ROW EXECUTE FUNCTION audit.log_event();
959 CREATE TRIGGER audit_roles_trigger AFTER INSERT OR UPDATE OR DELETE ON auth.roles FOR
960     ↪ EACH ROW EXECUTE FUNCTION audit.log_event();

```

```

952
953  -- =====
954  -- REVISED USER PERMISSIONS: 4-TIER + BOOTSTRAP ADMIN (ZERO TRUST)
955  -- Updated for BCNF Compliance (Views, Cities, Shipping Isolation)
956  -- =====
957
958  BEGIN;
959
960  -- 1. CLEANUP ROLES
961  DO $$
962  DECLARE
963      r_name text;
964      roles text[] := ARRAY['app_public', 'app_customer', 'app_employee', 'app_manager',
965      ↪ 'warehouse_bot', 'sec_auditor', 'app_admin'];
966  BEGIN
967      FOREACH r_name IN ARRAY roles
968      LOOP
969          IF EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname = r_name) THEN
970              EXECUTE 'REASSIGN OWNED BY ' || quote_ident(r_name) || ' TO postgres';
971              EXECUTE 'DROP OWNED BY ' || quote_ident(r_name);
972              EXECUTE 'DROP ROLE ' || quote_ident(r_name);
973          END IF;
974      END LOOP;
975  END $$;
976
977  -- 2. CREATE ROLES
978  -- TIER 0: SYSTEM BOOTSTRAP ADMIN (Restricted, not superuser)
979  CREATE USER app_admin WITH PASSWORD 'Admin_Bootstrap_X12!';
980
981  -- OPERATIONAL TIERS
982  CREATE USER app_public WITH PASSWORD 'Public_Access_77!';
983  CREATE USER app_customer WITH PASSWORD 'Shopper_Secure_88!';
984  CREATE USER app_employee WITH PASSWORD 'Staff_Worker_11!';
985  CREATE USER app_manager WITH PASSWORD 'Staff_Manager_99!';
986  CREATE USER warehouse_bot WITH PASSWORD 'Bot_Automated_00!';
987  CREATE USER sec_auditor WITH PASSWORD 'Audit_Eyes_Only!';
988
989  -- 3. UTILITY SCHEMA
990  CREATE SCHEMA IF NOT EXISTS utils;
991  GRANT USAGE ON SCHEMA utils TO public;
992
993  CREATE OR REPLACE FUNCTION utils.get_my_store_id() RETURNS INTEGER AS $$
994  DECLARE
995      v_uid INTEGER;
996      v_sid INTEGER;
997  BEGIN
998      BEGIN
999          v_uid := current_setting('app.current_user_id', true)::INTEGER;
1000      EXCEPTION WHEN OTHERS THEN

```

```

1000         RETURN NULL;
1001     END;
1002     IF v_uid IS NULL THEN RETURN NULL; END IF;
1003     SELECT store_id INTO v_sid FROM hr.employees WHERE user_id = v_uid;
1004     RETURN v_sid;
1005 END;
1006 $$ LANGUAGE plpgsql SECURITY DEFINER;
1007
1008 -- 4. BASELINE SECURITY (ZERO TRUST)
1009 REVOKE ALL ON SCHEMA public FROM PUBLIC;
1010 REVOKE ALL ON ALL FUNCTIONS IN SCHEMA auth FROM PUBLIC;
1011 REVOKE ALL ON ALL FUNCTIONS IN SCHEMA customers FROM PUBLIC;
1012 REVOKE ALL ON ALL FUNCTIONS IN SCHEMA inventory FROM PUBLIC;
1013 REVOKE ALL ON ALL FUNCTIONS IN SCHEMA sales FROM PUBLIC;
1014 REVOKE ALL ON ALL FUNCTIONS IN SCHEMA hr FROM PUBLIC;
1015 REVOKE ALL ON ALL FUNCTIONS IN SCHEMA audit FROM PUBLIC;
1016 REVOKE ALL ON FUNCTION utils.get_my_store_id FROM PUBLIC;
1017
1018 -- =====
1019 -- 5. GRANT PERMISSIONS
1020 -- =====
1021
1022 -- -----
1023 -- TIER 0: APP ADMIN (BOOTSTRAPPER)
1024 -- -----
1025 GRANT USAGE ON SCHEMA inventory, auth, hr TO app_admin;
1026
1027 -- Action 1: Check Store Existence
1028 GRANT SELECT ON inventory.stores TO app_admin;
1029
1030 -- Action 2: Check User Existence
1031 GRANT SELECT ON auth.users TO app_admin;
1032
1033 -- Action 3: Run Bootstrap Function
1034 GRANT EXECUTE ON FUNCTION inventory.bootstrap_new_store TO app_admin;
1035
1036 -- Action 4: Ensure Roles Exist
1037 GRANT SELECT, INSERT ON auth.roles TO app_admin;
1038 GRANT USAGE, SELECT ON SEQUENCE auth.roles_role_id_seq TO app_admin;
1039
1040 -- -----
1041 -- TIER 1: PUBLIC
1042 -- -----
1043 GRANT USAGE ON SCHEMA inventory, auth, customers TO app_public;
1044 GRANT SELECT ON inventory.products, inventory.categories, inventory.reviews TO
    ↪ app_public;
1045 GRANT SELECT ON inventory.product_categories TO app_public;
1046 GRANT SELECT ON customers.cities TO app_public;
1047

```

```

1048 GRANT EXECUTE ON FUNCTION auth.authenticate_user TO app_public;
1049 GRANT EXECUTE ON FUNCTION auth.register_customer TO app_public;
1050
1051 -----
1052 -- TIER 2: CUSTOMER
1053 -----
1054 GRANT app_public TO app_customer;
1055 GRANT USAGE ON SCHEMA sales, customers TO app_customer;
1056
1057 -- Tables
1058 GRANT SELECT, INSERT, UPDATE ON sales.orders, sales.order_items, sales.shopping_carts,
    ↪ sales.cart_items TO app_customer;
1059 GRANT SELECT, INSERT, UPDATE ON customers.addresses, customers.wishlist,
    ↪ customers.wishlist_items TO app_customer;
1060 GRANT SELECT, UPDATE ON customers.customers TO app_customer;
1061 GRANT INSERT, UPDATE, SELECT ON inventory.reviews TO app_customer;
1062 GRANT INSERT, SELECT ON sales.returns, sales.return_items TO app_customer;
1063 GRANT SELECT (user_id, email, first_name, last_name, phone_number) ON auth.users TO
    ↪ app_customer;
1064
1065 GRANT SELECT ON customers.v_full_addresses TO app_customer;
1066 GRANT SELECT ON sales.v_order_summary TO app_customer;
1067 GRANT INSERT ON customers.cities TO app_customer;
1068
1069 -- Sequences
1070 GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA sales, customers TO app_customer;
1071 GRANT USAGE, SELECT ON SEQUENCE inventory.reviews_review_id_seq TO app_customer;
1072
1073 -- Functions
1074 GRANT EXECUTE ON FUNCTION customers.add_address TO app_customer;
1075 GRANT EXECUTE ON FUNCTION customers.update_marketing_preferences TO app_customer;
1076 GRANT EXECUTE ON FUNCTION customers.create_wishlist TO app_customer;
1077 GRANT EXECUTE ON FUNCTION customers.add_item_to_wishlist TO app_customer;
1078 GRANT EXECUTE ON FUNCTION inventory.submit_review TO app_customer;
1079 GRANT EXECUTE ON FUNCTION sales.add_to_cart TO app_customer;
1080 GRANT EXECUTE ON FUNCTION sales.request_return TO app_customer;
1081 GRANT EXECUTE ON FUNCTION inventory.update_stock_level TO app_customer;
1082 GRANT EXECUTE ON FUNCTION sales.checkout_cart TO app_customer;
1083
1084 -----
1085 -- TIER 3: EMPLOYEE
1086 -----
1087 GRANT USAGE ON SCHEMA inventory, sales, auth TO app_employee;
1088 GRANT EXECUTE ON FUNCTION utils.get_my_store_id TO app_employee;
1089
1090 -- View Catalog
1091 GRANT SELECT ON inventory.products, inventory.categories, inventory.product_categories
    ↪ TO app_employee;
1092

```

```

1093  -- Orders & Shipping
1094  GRANT SELECT, UPDATE ON sales.orders TO app_employee;
1095  GRANT SELECT ON sales.order_items TO app_employee;
1096  GRANT SELECT, INSERT, UPDATE ON sales.shipping TO app_employee;
1097
1098  GRANT SELECT ON sales.v_order_summary TO app_employee;
1099
1100  -- Auth
1101  GRANT EXECUTE ON FUNCTION auth.authenticate_user TO app_employee;
1102
1103  GRANT USAGE, SELECT ON SEQUENCE sales.shipping_shipping_id_seq TO app_employee;
1104
1105  GRANT EXECUTE ON FUNCTION sales.add_shipping_details TO app_employee;
1106  GRANT EXECUTE ON FUNCTION sales.update_order_status TO app_employee;
1107  GRANT EXECUTE ON FUNCTION inventory.update_stock_level TO app_employee;
1108
1109  -- -----
1110  -- TIER 4: MANAGER
1111  -- -----
1112  GRANT app_employee TO app_manager;
1113  GRANT USAGE ON SCHEMA hr, customers TO app_manager;
1114  GRANT EXECUTE ON FUNCTION utils.get_my_store_id TO app_manager;
1115
1116  -- HR
1117  GRANT SELECT, INSERT, UPDATE ON hr.employees TO app_manager;
1118  GRANT SELECT (user_id, email, first_name, last_name, phone_number, created_at,
1119    ↪ is_active, user_type) ON auth.users TO app_manager;
1120
1121  -- Inventory
1122  GRANT INSERT, UPDATE ON inventory.products TO app_manager;
1123  GRANT INSERT ON inventory.categories TO app_manager;
1124  GRANT INSERT, UPDATE, DELETE ON inventory.product_categories TO app_manager;
1125  GRANT SELECT, INSERT, UPDATE ON inventory.discounts TO app_manager;
1126
1127  -- Sales
1128  GRANT SELECT, UPDATE ON sales.returns TO app_manager;
1129  GRANT EXECUTE ON FUNCTION sales.process_return TO app_manager;
1130  GRANT EXECUTE ON FUNCTION inventory.get_store_performance TO app_manager;
1131
1132  -- Functions
1133  GRANT EXECUTE ON FUNCTION auth.register_employee TO app_manager;
1134  GRANT EXECUTE ON FUNCTION inventory.add_product TO app_manager;
1135  GRANT EXECUTE ON FUNCTION inventory.create_discount TO app_manager;
1136  GRANT EXECUTE ON FUNCTION inventory.create_category TO app_manager;
1137
1138  GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA inventory, hr TO app_manager;
1139
1140  -- -----
1141  -- TIER 5: WAREHOUSE BOT

```

```

1141  -- -----
1142  GRANT USAGE ON SCHEMA inventory, sales TO warehouse_bot;
1143  GRANT SELECT, UPDATE ON inventory.products TO warehouse_bot;
1144  GRANT SELECT ON sales.orders, sales.order_items TO warehouse_bot;
1145  GRANT SELECT, UPDATE ON sales.shipping TO warehouse_bot;
1146
1147  GRANT EXECUTE ON FUNCTION sales.add_shipping_details TO warehouse_bot;
1148  GRANT EXECUTE ON FUNCTION inventory.update_stock_level TO warehouse_bot;
1149
1150  -- -----
1151  -- TIER 6: AUDITOR
1152  -- -----
1153  GRANT USAGE ON SCHEMA audit TO sec_auditor;
1154  GRANT SELECT ON audit.audit_log TO sec_auditor;
1155  GRANT EXECUTE ON FUNCTION audit.get_user_activity TO sec_auditor;
1156
1157  -- =====
1158  -- 6. STORE ISOLATION POLICIES (RLS)
1159  -- =====
1160
1161  -- Enable RLS
1162  ALTER TABLE sales.orders ENABLE ROW LEVEL SECURITY;
1163  ALTER TABLE sales.order_items ENABLE ROW LEVEL SECURITY;
1164  ALTER TABLE sales.shipping ENABLE ROW LEVEL SECURITY;
1165  ALTER TABLE customers.customers ENABLE ROW LEVEL SECURITY;
1166  ALTER TABLE customers.addresses ENABLE ROW LEVEL SECURITY;
1167  ALTER TABLE customers.wishlist ENABLE ROW LEVEL SECURITY;
1168  ALTER TABLE customers.wishlist_items ENABLE ROW LEVEL SECURITY;
1169  ALTER TABLE inventory.reviews ENABLE ROW LEVEL SECURITY;
1170  ALTER TABLE auth.users ENABLE ROW LEVEL SECURITY;
1171  ALTER TABLE hr.employees ENABLE ROW LEVEL SECURITY;
1172  ALTER TABLE inventory.products ENABLE ROW LEVEL SECURITY;
1173
1174  -- DROP EXISTING POLICIES
1175  DROP POLICY IF EXISTS customer_isolation_orders ON sales.orders;
1176  DROP POLICY IF EXISTS staff_view_store_orders ON sales.orders;
1177  DROP POLICY IF EXISTS bot_view_all_orders ON sales.orders;
1178  DROP POLICY IF EXISTS customer_view_own_shipping ON sales.shipping;
1179  DROP POLICY IF EXISTS staff_view_store_shipping ON sales.shipping;
1180  DROP POLICY IF EXISTS bot_view_all_shipping ON sales.shipping;
1181  DROP POLICY IF EXISTS public_view_products ON inventory.products;
1182  DROP POLICY IF EXISTS manager_manage_store_products ON inventory.products;
1183  DROP POLICY IF EXISTS manager_manage_store_staff ON hr.employees;
1184  DROP POLICY IF EXISTS employee_view_self ON hr.employees;
1185  DROP POLICY IF EXISTS manager_view_users ON auth.users;
1186  DROP POLICY IF EXISTS admin_view_users ON auth.users;
1187  DROP POLICY IF EXISTS user_view_self ON auth.users;
1188  DROP POLICY IF EXISTS customer_isolation_profiles ON customers.customers;
1189  DROP POLICY IF EXISTS customer_isolation_addresses ON customers.addresses;

```



```

1190 DROP POLICY IF EXISTS customer_isolation_wishlist ON customers.wishlist;
1191 DROP POLICY IF EXISTS customer_isolation_wishlist_items ON customers.wishlist_items;
1192 DROP POLICY IF EXISTS view_reviews ON inventory.reviews;
1193 DROP POLICY IF EXISTS manage_own_reviews ON inventory.reviews;
1194
1195 -- [SALES.ORDERS]
1196 CREATE POLICY customer_isolation_orders ON sales.orders FOR ALL TO app_customer
1197     USING (customer_id = current_setting('app.current_user_id', true)::INTEGER);
1198
1199 CREATE POLICY staff_view_store_orders ON sales.orders FOR ALL TO app_employee,
1200     ↪ app_manager
1201     USING (store_id = utils.get_my_store_id());
1202
1203 CREATE POLICY bot_view_all_orders ON sales.orders FOR ALL TO warehouse_bot USING
1204     ↪ (true);
1205
1206 -- [SALES.SHIPPING]
1207 -- Customers see shipping for their own orders
1208 CREATE POLICY customer_view_own_shipping ON sales.shipping FOR SELECT TO app_customer
1209     USING (order_id IN (
1210         SELECT order_id FROM sales.orders
1211         WHERE customer_id = current_setting('app.current_user_id', true)::INTEGER
1212     ));
1213
1214 -- Staff see shipping for orders from their store
1215 CREATE POLICY staff_view_store_shipping ON sales.shipping FOR ALL TO app_employee,
1216     ↪ app_manager
1217     USING (order_id IN (
1218         SELECT order_id FROM sales.orders
1219         WHERE store_id = utils.get_my_store_id()
1220     ));
1221
1222 CREATE POLICY bot_view_all_shipping ON sales.shipping FOR ALL TO warehouse_bot USING
1223     ↪ (true);
1224
1225 -- [INVENTORY.PRODUCTS]
1226 CREATE POLICY public_view_products ON inventory.products FOR SELECT TO PUBLIC USING
1227     ↪ (true);
1228
1229 CREATE POLICY manager_manage_store_products ON inventory.products FOR ALL TO
1230     ↪ app_manager
1231     USING (store_id = utils.get_my_store_id())
1232     WITH CHECK (store_id = utils.get_my_store_id());
1233
1234 -- [HR.EMPLOYEES]
1235 CREATE POLICY manager_manage_store_staff ON hr.employees FOR ALL TO app_manager
1236     USING (store_id = utils.get_my_store_id());
1237
1238 CREATE POLICY employee_view_self ON hr.employees FOR SELECT TO app_employee

```

```

1233     USING (user_id = current_setting('app.current_user_id', true)::INTEGER);
1234
1235 -- [AUTH.USERS]
1236 CREATE POLICY manager_view_users ON auth.users FOR SELECT TO app_manager USING (true);
1237 CREATE POLICY admin_view_users ON auth.users FOR SELECT TO app_admin USING (true);
1238 CREATE POLICY user_view_self ON auth.users FOR SELECT TO public
1239     USING (user_id = current_setting('app.current_user_id', true)::INTEGER);
1240
1241 -- [CUSTOMERS & WISHLISTS]
1242 CREATE POLICY customer_isolation_profiles ON customers.customers FOR ALL TO
1243     ↪ app_customer
1244     USING (user_id = current_setting('app.current_user_id', true)::INTEGER);
1245
1246 CREATE POLICY customer_isolation_addresses ON customers.addresses FOR ALL TO
1247     ↪ app_customer
1248     USING (customer_id = current_setting('app.current_user_id', true)::INTEGER);
1249
1250 CREATE POLICY customer_isolation_wishlist ON customers.wishlist FOR ALL TO
1251     ↪ app_customer
1252     USING (customer_id = current_setting('app.current_user_id', true)::INTEGER);
1253
1254 CREATE POLICY customer_isolation_wishlist_items ON customers.wishlist_items FOR ALL TO
1255     ↪ app_customer
1256     USING (wishlist_id IN (
1257         SELECT wishlist_id FROM customers.wishlist
1258         WHERE customer_id = current_setting('app.current_user_id', true)::INTEGER
1259     ));
1260
1261 -- [REVIEWS]
1262 CREATE POLICY view_reviews ON inventory.reviews FOR SELECT TO public USING (true);
1263 CREATE POLICY manage_own_reviews ON inventory.reviews FOR ALL TO app_customer
1264     USING (customer_id = current_setting('app.current_user_id', true)::INTEGER);
1265
1266 -- Enforce Policies
1267 ALTER TABLE sales.orders FORCE ROW LEVEL SECURITY;
1268 ALTER TABLE sales.shipping FORCE ROW LEVEL SECURITY;
1269 ALTER TABLE hr.employees FORCE ROW LEVEL SECURITY;
1270 ALTER TABLE inventory.products FORCE ROW LEVEL SECURITY;
1271 ALTER TABLE auth.users FORCE ROW LEVEL SECURITY;
1272
1273 -- 7. FUNCTION SECURITY ESCALATION (SECURITY DEFINER)
1274 -- =====
1275 ALTER FUNCTION inventory.update_stock_level(INTEGER, INTEGER) SECURITY DEFINER;
1276 ALTER FUNCTION sales.checkout_cart(INTEGER, VARCHAR, INTEGER) SECURITY DEFINER;
1277 ALTER FUNCTION auth.register_customer(VARCHAR, VARCHAR, VARCHAR, VARCHAR) SECURITY
1278     ↪ DEFINER;
1279
1280 COMMIT;

```

F AI Declaration

Tool Used	Purpose	Outcome/Refinement
Gemini	Generation of LaTeX boilerplate, PL/pgSQL syntax checking, and debugging.	Output was heavily modified to match specific constraints; SQL was manually refactored to include RLS policies. Sources were verified against academic databases.
Gemini	Learning advanced SQL concepts (RLS, Triggers) and evaluating draft work against the assignment requirements.	AI explanations were used to debug logic errors. Feedback on the draft was used to refine the "Critical Evaluation" section, ensuring all learning outcomes were addressed.